

From Characters to Transformers

Next-token prediction, attention, and a decoder-only language model

Prof. Nipun Batra

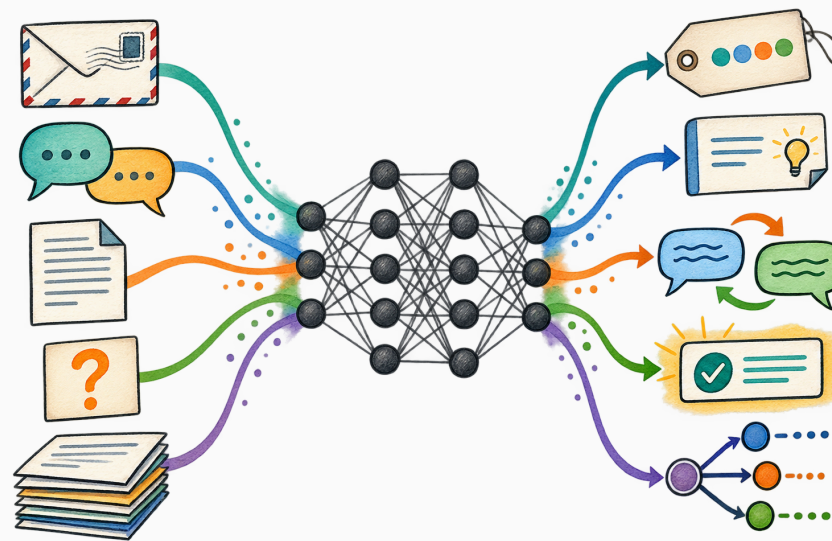
ES 667 · Deep Learning · IIT Gandhinagar

Lecture 1 · Next-token prediction

How can a neural network work with **language**?

CLASSIFY
UNDERSTAND
COMPRESS
TRANSLATE
ANSWER
GENERATE

email → spam?
review → sentiment
article → summary
English → Hindi
context + question → answer
prompt → continuation



Examples: MIT 6.S191 L2 and Stanford CME 295 L1.

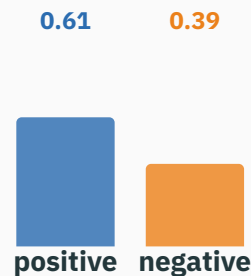
One input → one label

REVIEW

“The acting was wonderful, but the ending was disappointing.”

TEXT INPUT

review text



We have seen classification before. What changes when the input is text?

Intent classification

UTTERANCE

Book a flight to Delhi

UTTERANCE

What is my account balance?

UTTERANCE

Cancel my booking

LABEL



BOOK_FLIGHT

LABEL



CHECK_BALANCE

LABEL



CANCEL_BOOKING

Text → one class label, whatever the input length.

Token-level prediction: named entities

Sundar	Pichai	visited	IIT	Gandhinagar	on	Monday.
PERSON	PERSON	O	ORG	ORG	O	DATE

INPUT

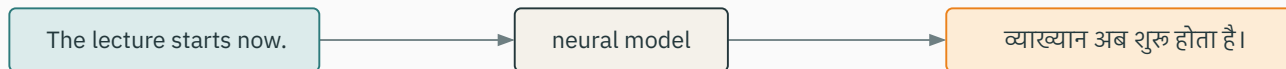
OUTPUT

a sequence of T tokens

one class label for each of the T tokens

One token in, one label out.

Translation changes the sequence length



ENGLISH

HINDI

4 displayed words

5 displayed words

One input word does not always correspond to one output word.

Summarize a notice

IITGN NOTICE



The library will remain open until midnight during examinations.
Extended hours begin Monday.
Entry requires a valid institute ID.
The ground-floor reading room stays open.
Group rooms close at 10 pm.
Regular timings resume after examinations.

ONE-SENTENCE SUMMARY

During exams, IITGN's library and ground-floor reading room stay open until midnight from Monday; bring institute ID.

How does the model decide what matters?

Answer a question using the context

CONTEXT

IIT Gandhinagar's permanent campus is located in Palaj, on the banks of the Sabarmati River.

QUESTION

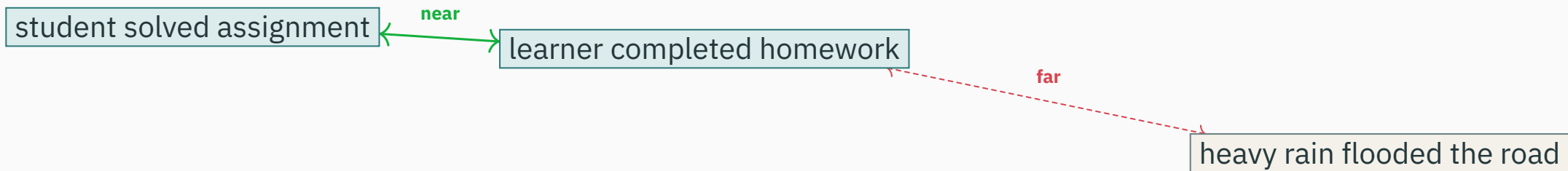
Where is IITGN's permanent campus?

ANSWER

Palaj

For this question, we need the location. Another question may need a different part of the context.

Represent similar meanings with nearby vectors



Can similar sentences have nearby vectors?

Generation has many valid answers

PROMPT

The student opened the laptop and ...

A

started coding.

C

joined class.

B

checked email.

D

saw a warning.



Several answers make sense. What should the model output?

Generation is still classification

PREFIX

The student opened the laptop and

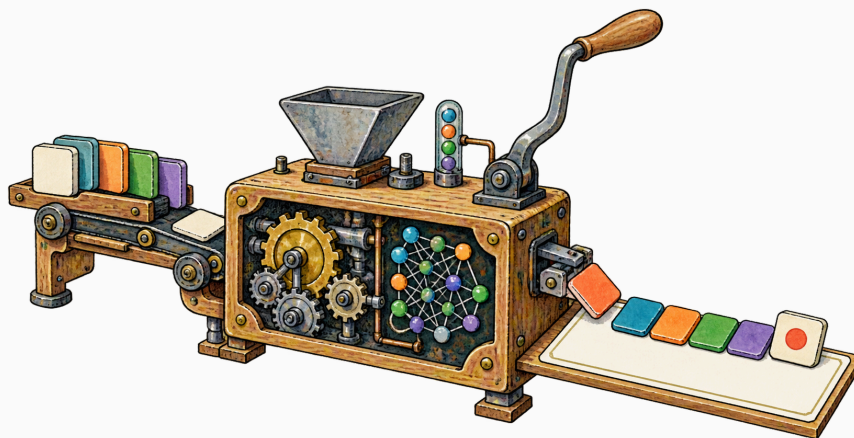


A probability for each possible next token.

Generate a name

Start with a small dataset.

**Can a neural network
generate Indian names?**



aabid

aarav

avni

naveen

vani

zeel

Worked example: Nipun Batra, Next-Token Prediction.

What exactly should we predict?

CHARACTER

transformer → t | r | a | ...

WORD

deep learning → deep | learning

SUBWORD

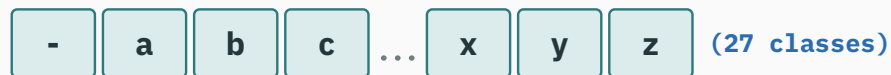
unbelievable → un | believ | able

A token is one unit the model predicts.

Start with one character per token. We can write out every training example and vector.

The task: generate an Indian name

VOCABULARY



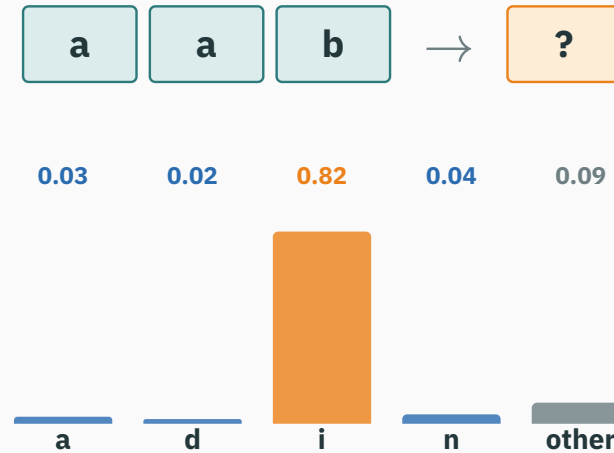
$$\mathcal{V} = \{-, a, \dots, z\}, \quad |\mathcal{V}| = 27$$

BOUNDARY SYMBOL

Put - before a name to fill the context, and after it to mark the end. In this example, the same symbol handles BOS, padding, and EOS.

Learn a probability distribution over 27 possible next characters.

Generation is repeated 27-class classification



$$f_{\theta}(a, a, b) = p_{\theta}(c_{\text{next}} \mid a, a, b)$$

Choose a context length

How much history does the first model receive?

$$w = 1$$

one previous character

$$w = 3$$

three previous characters

$$w = \text{ALL}$$

a variable-length prefix

Use $w = 3: p_{\theta}(t_i \mid t_{i-3:i-1})$.

Here t_i is the token ID at position i . Three earlier IDs give the MLP a fixed-size input.

Construct the training examples

pair	1	2	3	4	5	6
context	---	--a	-aa	aab	abi	bid
target	a	a	b	i	d	-

Six contexts, six next-character targets.

Predict the five characters in the name, then predict - after d to end it.

Slide the context window

NAME WITH BOUNDARY SYMBOLS

- - - a a b i d -

WINDOW 1



SHIFT
RIGHT



SHIFT
RIGHT



Later, causal self-attention will let us predict these targets in parallel during training.

Repeat for every name

ONE NAME

length L
 L characters + one EOS target



EXAMPLES

$L + 1$ context-target pairs

$$N_{\text{examples}} = \sum_{n=1}^N (L_n + 1)$$

Split the names into training, validation, and test sets **before** making the windows.

How do we turn characters into numbers?

$$\text{a a b} \rightarrow E[\cdot] \rightarrow [e_a, e_a, e_b] \xrightarrow{\text{MLP}} 27_{\text{logits}} \xrightarrow{\text{softmax}} \text{CE}(i)$$

CHARACTERS

a, b, i: categorical symbols

MLP INPUT

a row vector of numbers

? $\rightarrow$

Give each character a one-hot vector.

One-hot encoding

$$\text{a a b} \rightarrow E[\cdot] \rightarrow [e_a, e_a, e_b] \xrightarrow{\text{MLP}} 27_{\text{logits}} \xrightarrow{\text{softmax}} \text{CE(i)}$$

token / ID	0	1	2	9	...	26
a	0	1	0	0	...	0
b	0	0	1	0	...	0
i	0	0	0	1	...	0

$$\underline{o_a o_b^T = o_a o_i^T = o_b o_i^T = 0}$$

Every pair of different characters has dot product zero. These vectors identify characters; they do not tell us which characters are similar.

Learn an embedding for each character

$$a \ a \ b \rightarrow E[\cdot] \rightarrow [e_a, e_a, e_b] \xrightarrow{\text{MLP}} 27_{\text{logits}} \xrightarrow{\text{softmax}} \text{CE}(i)$$

EMBEDDING TABLE

$$E \in \mathbb{R}^{|\mathcal{V}| \times d} = \mathbb{R}^{27 \times 2}$$

row	dim 1	dim 2
-	0.1	-0.4
a	1.2	0.1
b	0.2	1.1
...	...	...
z	-0.3	0.6

LOOKUP RULE

$$e_c = E[\text{id}(c)]$$

one $1 \times d$ embedding row for character c

LEARNABLE

Backpropagation updates the rows to help predict the next token.

Look up the embedding

a a b $\rightarrow E[\cdot] \rightarrow [e_a, e_a, e_b] \rightarrow \text{MLP} \rightarrow 27_{\text{logits}} \rightarrow \text{softmax} \rightarrow \text{CE}(i)$

token	id	selected row
a	1	$E[1] = (1.2, 0.1)$
b	2	$E[2] = (0.2, 1.1)$
i	9	$E[9] = (-0.8, 0.9)$

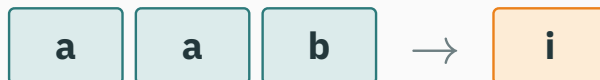
PyTorch

```
import torch
embed = torch.nn.Embedding(27, 2)
ids = torch.tensor([1, 1, 2])
rows = embed(ids)  # shape: [3, 2]
```

Each ID selects one trainable row.

Check the shapes

a a b $\rightarrow E[\cdot] \rightarrow [e_a, e_a, e_b] \xrightarrow{\text{MLP}} 27_{\text{logits}} \xrightarrow{\text{softmax}} \text{CE}(i)$



INTEGER INPUT

$\text{ids} = (t_1, t_2, t_3) = (1, 1, 2)$
shape: $(3,)$

EMBEDDINGS

$E[\text{ids}] = \begin{pmatrix} 1.2 & 0.1 \\ 1.2 & 0.1 \\ 0.2 & 1.1 \end{pmatrix}$
shape: 3×2

At position i , $e_i = E[t_i]$: one embedding row per token.

For a a b: $e_1 = e_a, e_2 = e_a, e_3 = e_b$.

Concatenate the embeddings

$$a \ a \ b \rightarrow E[\cdot] \rightarrow [e_a, e_a, e_b] \rightarrow \text{MLP} \rightarrow 27_{\text{logits}} \rightarrow \text{softmax} \rightarrow \text{CE}(i)$$

$$\begin{pmatrix} 1.2 & 0.1 \\ 1.2 & 0.1 \\ 0.2 & 1.1 \end{pmatrix} \rightarrow a_0 = [1.2, 0.1, 1.2, 0.1, 0.2, 1.1]$$

POSITION 1 · A

[1.2, 0.1]

POSITION 2 · A

[1.2, 0.1]

POSITION 3 · B

[0.2, 1.1]

$$a_0 = [e_1, e_2, e_3] \in \mathbb{R}^{1 \times wd} = \mathbb{R}^{1 \times 6}$$

Why concatenate?

$$a \ a \ b \rightarrow E[\cdot] \rightarrow [e_a, e_a, e_b] \rightarrow \text{MLP} \rightarrow 27_{\text{logits}} \rightarrow \text{softmax} \rightarrow \text{CE}(i)$$

ORDER 1

$$a \ b \ i \rightarrow [e_a, e_b, e_i]$$

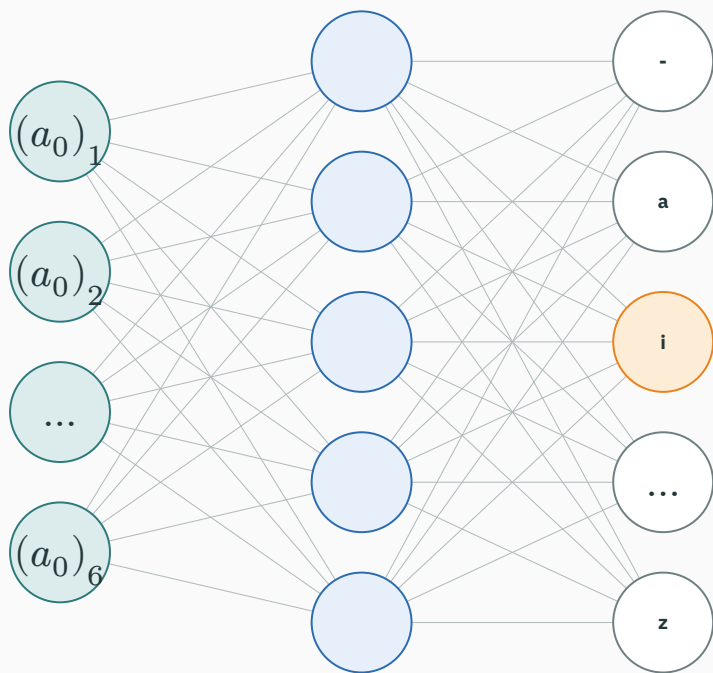
ORDER 2

$$i \ b \ a \rightarrow [e_i, e_b, e_a]$$

$$[e_a, e_b, e_i] \neq [e_i, e_b, e_a]$$

Adding gives the same result in both orders: $e_a + e_b + e_i = e_i + e_b + e_a$. Concatenation keeps each position in a separate part of the vector.

Pass the vector through an MLP



ONE HIDDEN LAYER

$$a_1 = \sigma(a_0 W_1 + b_1)$$

$$z = a_1 W_2 + b_2$$

$$W_1 \in \mathbb{R}^{wd \times d_h}$$

$$W_2 \in \mathbb{R}^{d_h \times |\mathcal{V}|}$$

Row vectors multiply matrices on the right. Here $wd = 6$, $|\mathcal{V}| = 27$; d_h is the hidden width.

Why exactly 27 outputs?

$a \ a \ b \rightarrow E[\cdot] \rightarrow [e_a, e_a, e_b] \rightarrow \text{MLP} \rightarrow \text{27 logits} \rightarrow \text{softmax} \rightarrow \text{CE}(i)$

- a b c ... x y z (27 classes)

$$z = a_1 W_2 + b_2 \in \mathbb{R}^{1 \times 27}$$

LOGIT z_{-}

score for boundary

LOGIT z_A

score for a

LOGIT z_I

score for i

One score for each possible next character.

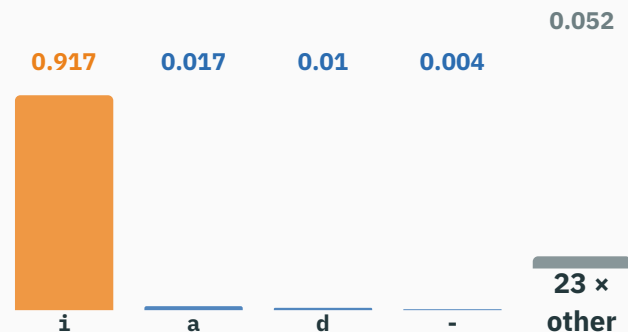
Turn the 27 scores into probabilities

$$a \ a \ b \rightarrow E[\cdot] \rightarrow [e_a, e_a, e_b] \rightarrow \text{MLP} \rightarrow 27 \text{ logits} \rightarrow \text{softmax} \rightarrow \text{CE}(i)$$

EXAMPLE LOGITS FOR AAB

$z_i = 5, \quad z_a = 1, \quad z_d = 0.5,$
 $z_- = -0.5$, and every other logit = -1 .

$$p_j = \frac{\exp(z_j)}{\sum_{r \in \mathcal{V}} \exp(z_r)}$$



All 27 probabilities are positive and sum to one. The other bar groups the remaining 23 characters.

Compute the loss

$$a, a, b \rightarrow E[\cdot] \rightarrow [e_a, e_a, e_b] \xrightarrow{\text{MLP}} 27_{\text{logits}} \xrightarrow{\text{softmax}} \text{CE}(i)$$

For target i : $\ell = -\log p_{\theta}(i \mid a, a, b)$

CONFIDENT AND CORRECT

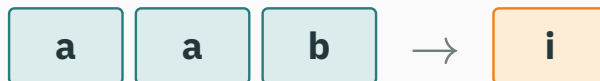
$$p(i) = 0.80$$
$$\ell = -\log(0.80) \approx 0.223$$

CONFIDENT AND WRONG

$$p(i) = 0.01$$
$$\ell = -\log(0.01) \approx 4.605$$

The more probability we give i , the smaller the loss.

The loss in PyTorch



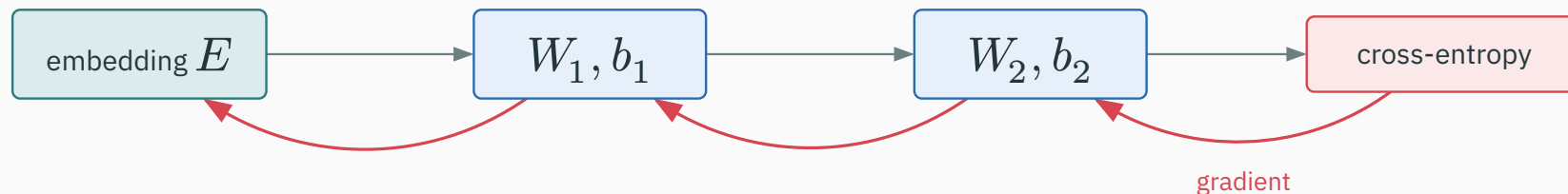
PyTorch

```
import torch
target = torch.tensor([9]) # token i; logits: [1, 27]
loss = torch.nn.functional.cross_entropy(logits, target)
loss.backward()
```

Pass the target ID to cross-entropy. Backpropagation computes gradients for the whole network.

Learn the embeddings and MLP weights

$$a \ a \ b \rightarrow E[\cdot] \rightarrow [e_a, e_a, e_b] \xrightarrow{\text{MLP}} 27_{\text{logits}} \xrightarrow{\text{softmax}} \text{CE}(i)$$

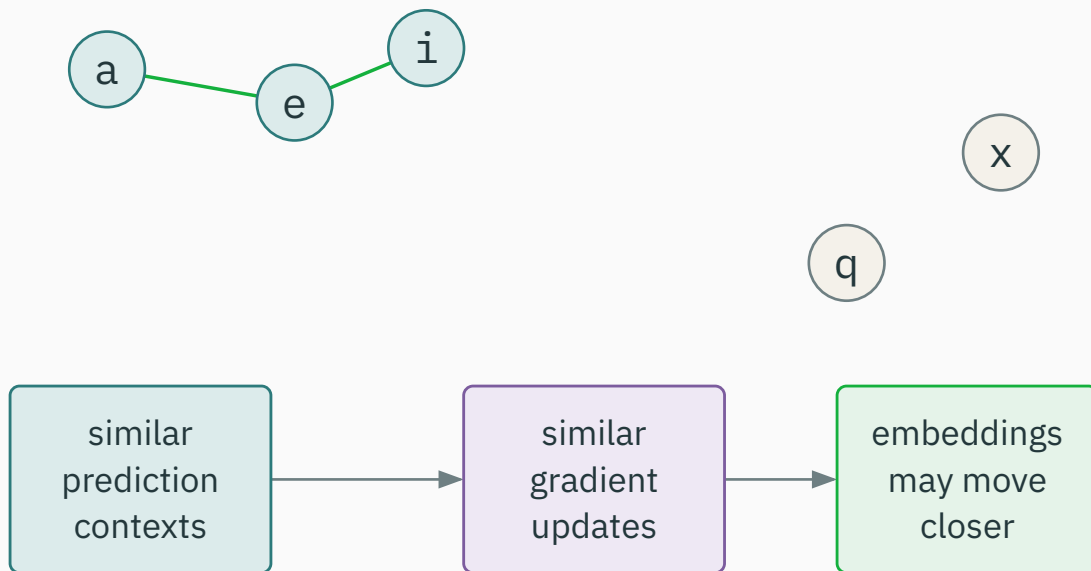


The same loss trains the embedding table and the MLP.

For $aab \rightarrow i$, we look up the rows for a and b , so those rows receive gradients. The target i does not cause an embedding lookup.

When could two embeddings become similar?

$a \ a \ b \rightarrow E[\cdot] \rightarrow [e_a, e_a, e_b] \rightarrow \text{MLP} \rightarrow 27_{\text{logits}} \rightarrow \text{softmax} \rightarrow \text{CE}(i)$



Sample, append, repeat

$$a \ a \ b \rightarrow E[\cdot] \rightarrow [e_a, e_a, e_b] \rightarrow \text{MLP} \rightarrow 27 \text{ logits} \rightarrow \text{softmax} \rightarrow \text{CE}(i)$$

START



SHIFT +
SAMPLE



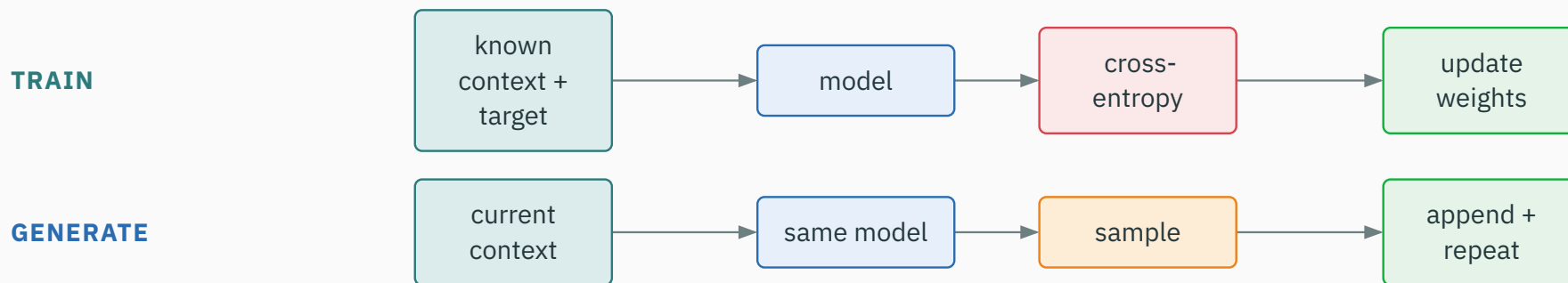
SHIFT +
SAMPLE



aab $\rightarrow$ i $\rightarrow$ abi $\rightarrow$ d $\rightarrow$ bid $\rightarrow$ -

We sampled -, so stop. The generated name is aabid.

Training and generation



Training uses the observed next token. Generation samples one.

Our character language model

$$p_{\theta}(t_i \mid t_{i-3:i-1})$$

SIZE

27 symbols and three context characters

CHARACTER-LEVEL

one character is one token

LANGUAGE MODEL

predicts the next token from previous tokens

Much larger generative models use this same next-token objective.

Neural language model: Bengio et al. (2003) · worked example: Next-Token Prediction.

Why begin with characters?

SENTENCE

deep learning is amazing

d | e | e | p | ^{SP} | l | e | a | r | n | i | n | g | ^{SP} | i | s | ...

SMALL VOCABULARY

we can write out every lookup and target

LONG SEQUENCES

even an ordinary sentence has many tokens

Character tokens

transformer → t | r | a | n | s | f | o | r | m | e | r

VOCABULARY

tiny: letters, digits, punctuation,
bytes

SEQUENCE

very long

UNKNOWN TEXT

almost none at byte/Unicode-unit
level

Small units cover many strings. They also produce long sequences.

Word tokens

deep learning is useful → deep | learning | is | useful

ONE ROOT, MANY VOCABULARY ENTRIES

walk · walks · walked · walking · walker · walkability

SHORT SEQUENCES

fewer tokens; words are easy to interpret

LARGE VOCABULARY

many entries; unseen words need handling

What happens to an unseen word?



These different words all become <UNK>.

Can we split text at spaces?

MIXED TEXT

IITGN में आज transformer lecture है :)

URL

<https://iitgn.ac.in/events?id=42>

CODE

```
scores[i] += q @ k.T
```

SCRIPTS

Devanagari + Latin

PUNCTUATION

`://?=[ ]@`

EMOJI

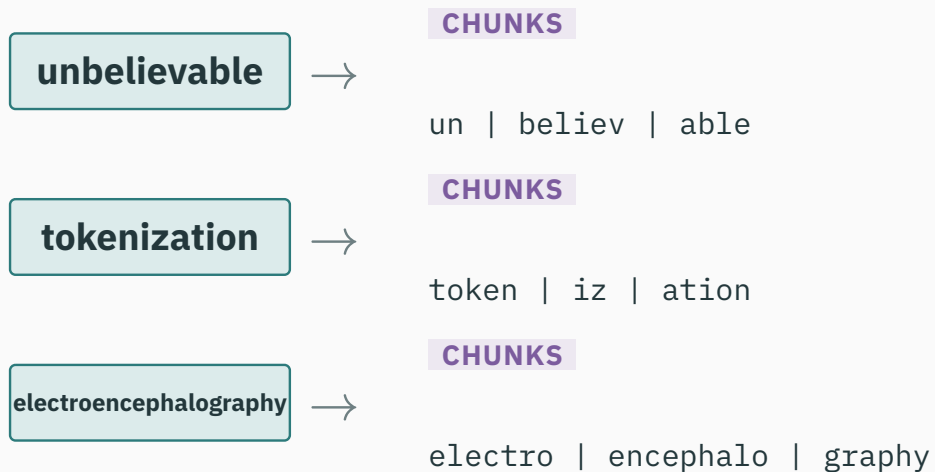
`:)` or Unicode

WHITESPACE

not a universal boundary

What counts as a word in each example?

Subword tokens



Reuse common pieces to represent both frequent and rare strings. We will leave the BPE training algorithm for later.

Choose the token size

CHARACTERS

tiny $|\mathcal{V}|$

long T



SUBWORDS

moderate $|\mathcal{V}|$

moderate T



WORDS

huge $|\mathcal{V}|$

short T

LARGER UNITS

more meaning per displayed token

SMALLER UNITS

easier to represent unfamiliar strings

Tokenization comparison: Stanford CME 295 L1 · MIT 6.S191 L2.

Compare the trade-off

unit	$ \mathcal{V} $	T	unseen text	in practice
character	small	long	excellent	meaning spans many steps
subword	moderate	moderate	strong	balances $ \mathcal{V} $ and T
word	huge	short	brittle	easy to read

Tokenization changes vocabulary size $|\mathcal{V}|$ and sequence length T .

We will use T again in attention: its score matrix has shape $T \times T$.

Tokens become integer IDs

token	id
The	17
cat	42
sat	91
on	8
the	5
mat	73

TOKEN SEQUENCE

The | cat | sat | on | the | mat

ID SEQUENCE

ids = [17, 42, 91, 8, 5, 73]

An ID labels a row in the learned embedding table.

Use token IDs in the model

$$t_1, t_2, \dots, t_T$$

$$t_i \in \{0, 1, \dots, |\mathcal{V}| - 1\}$$

DISPLAY

We will often show whole words.

MODEL INPUT

Each displayed unit has a token ID.

OBJECTIVE

Predict the next character, subword, or word.

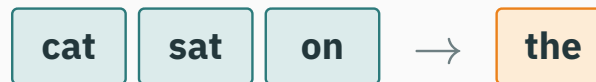
Change the tokens, keep the model

CHARACTER MODEL



embedding $\rightarrow$ concatenate $\rightarrow$ MLP $\rightarrow$ softmax

TOKEN MODEL



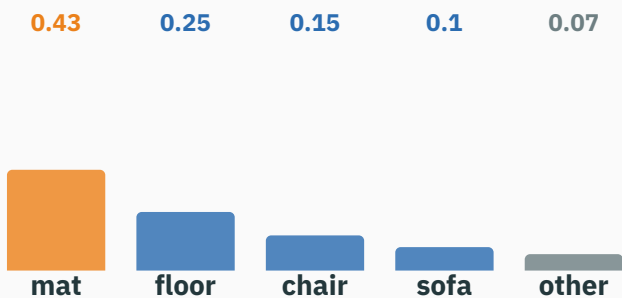
embedding $\rightarrow$ concatenate $\rightarrow$ MLP $\rightarrow$ softmax

We changed the vocabulary, the tokenizer, and the training data.

What could come next?

PREFIX

The cat sat on the ___



The training text gives us one target at each position. The model can give probability to several reasonable continuations.

Construct examples from a sentence

visible prefix	next target
<BOS>	deep
deep	learning
deep learning	is
deep learning is	fun
deep learning is fun	<EOS>

Keep the whole prefix this time. It grows by one token per row.

Here $t_1 = \text{id}(<\text{BOS}>)$ and $t_T = \text{id}(<\text{EOS}>)$. At position i , predict t_{i+1} .

Multiply the next-token probabilities

deep learning is fun

$$\begin{aligned} & p(\text{deep learning is fun}) \\ &= p(\text{deep}) \\ &\quad \times p(\text{learning} \mid \text{deep}) \\ &\quad \times p(\text{is} \mid \text{deep learning}) \\ &\quad \times p(\text{fun} \mid \text{deep learning is}). \end{aligned}$$

The chain rule gives this product exactly. We still need a model to turn each prefix into a prediction.

How should we represent the context?

$$\mathcal{L}(\theta) = - \sum_{i=1}^{T-1} \log p_{\theta}(t_{i+1} \mid t_{1:i})$$

TRAINING OBJECTIVE

Increase the probability of the observed next token at every position.

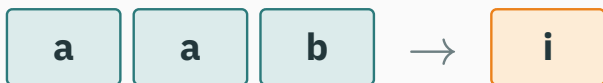
MODEL DESIGN

How do we represent a prefix that keeps getting longer?

The same loss can train different ways of representing the context.

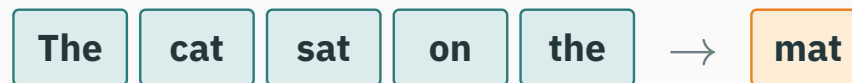
What if the context keeps growing?

THREE CHARACTERS



$(3 \times d \text{ numbers} \rightarrow \text{concatenate})$

GROWING PREFIX



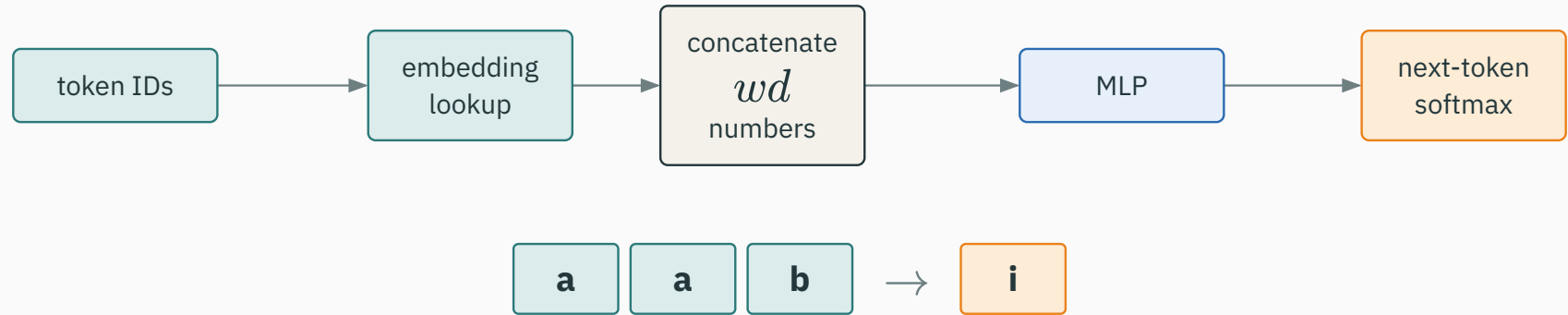
$(\text{length can keep growing})$

How can the model use a longer context?

Next, compare concatenation and averaging. Then let the model choose which parts of the context to use for each query.

Lecture 2: How attention works

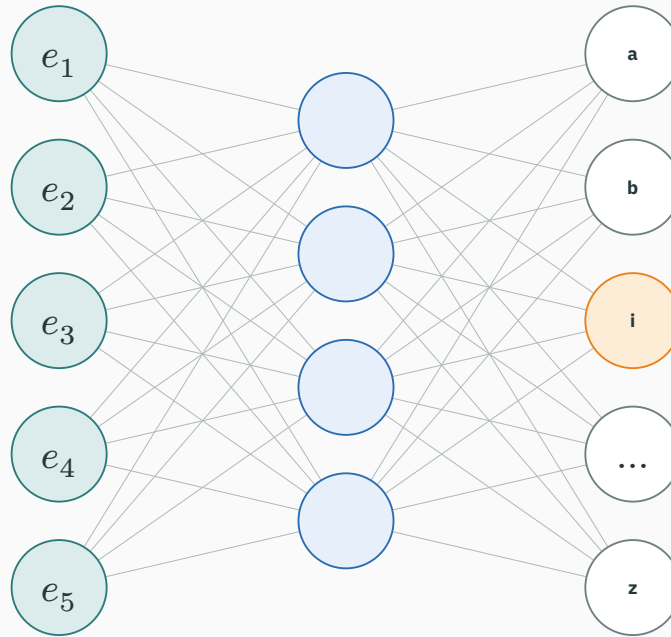
The fixed-window pipeline



Our window has $w = 3$ tokens. What changes if we use five to predict the next one?

What if the window is five tokens?

Five tokens $\rightarrow$ five embedding vectors $\rightarrow$ one wider first layer



The first layer now takes $5d$ inputs instead of $3d$.

What if the window is 100 tokens?

REPRESENTATION

$$a_0 = [e_1, e_2, \dots, e_{100}] \in \mathbb{R}^{1 \times 100d}$$

The input is now **100 times** the embedding width.

FIRST MLP LAYER

$$W_1 \in \mathbb{R}^{100d \times d_h}$$

parameter count: $100dd_h + d_h$

If $d = 256$ and hidden width $d_h = 1024$, this layer has about 26.2 million weights.

What if the window is 10,000 tokens?



10,000 embedding slots, with most omitted here

WIDTH

$10,000d$ input coordinates

PARAMETERS

$10,000dd_h$ first-layer weights

FIXED SIZE

the network expects exactly
10,000 slots

The window is part of the architecture

WINDOW $w = 3$

WINDOW $w = 5$

WINDOW $w = 100$

$$W_1 \in \mathbb{R}^{3d \times d_h}$$

$$W_1 \in \mathbb{R}^{5d \times d_h}$$

$$W_1 \in \mathbb{R}^{100d \times d_h}$$

Change the window, and we must change the network.

Which context would help after bank?

Both prefixes end in the same word. What information differs?

FINANCE CONTEXT

She deposited **money** in the **bank** ___

RIVER CONTEXT

She watched **water** by the **bank** ___

| bank is already an input. We want to predict the token **after it**.

| The earlier words tell us about different situations. A useful predictor should be able to use that difference.

The useful clue may lie outside the window

She deposited **money** after a long wait near the bank ___

A THREE-TOKEN WINDOW SEES

near the bank

The finance-related clue **money** is outside this window.

WHAT WE WANT TO RETAIN

Information from **money** and **deposited**, even when other words intervene.

Distance alone does not tell us what matters.

Gather information from the allowed prefix without concatenating every embedding into a wider input.

Context cannot supply a clue that is missing

She waited near the bank ___

A FINANCIAL INSTITUTION?

That is possible.

THE SIDE OF A RIVER?

That is also possible.

| This prefix does not settle the meaning. A context mechanism can use available evidence; it cannot manufacture missing evidence.

| Next, use two prefixes that do provide different clues.

Same lookup embedding, different context

FINANCE CONTEXT

She deposited **money** in the **bank** ____

RIVER CONTEXT

She watched **water** by the **bank** ____

$$e_6 = E[t_6] = e_{\text{bank}} \text{ in both prefixes}$$

The lookup uses the token ID, not its neighbours. Both occurrences start with the same bank embedding.

DESIRED FINANCE-CONTEXT RESULT

A representation informed by `money` and `deposited`.

DESIRED RIVER-CONTEXT RESULT

A representation informed by `water` and `watched`.

Desired behaviour, not outputs measured from our untrained toy model.

Keep the embedding; add context

She deposited money in the **bank** ---

$e_6 = E[t_6]$: the familiar lookup embedding for bank at position 6.

$$\begin{array}{ccccc} e_6 & & \Delta e_6 & & e'_6 \\ \text{lookup embedding} & + & \text{update from context} & = & \text{context-enriched} \\ & & & & \text{embedding} \end{array}$$

What should we add to bank? Information from money and deposited can help this occurrence reflect its finance context.

$$e'_6 \rightarrow \text{vocabulary predictor} \rightarrow p(\text{next token})$$

The prime means after adding context, at the same position. This does not overwrite the table E .

What should context add to bank's embedding?

She deposited money in the **bank** ____

Use a toy embedding with **three coordinates**.

	coordinate 1	coordinate 2	coordinate 3
bank's embedding e_6	1	1	0
+ context update Δe_6	?	?	?
= enriched embedding e'_6	1 + ?	1 + ?	0 + ?

We add matching coordinates: **three numbers + three numbers**.

Each ? is an update number we still need to compute, not a predicted token.

| How can the **six input embeddings** supply these **three update numbers**?

First try: average the six input embeddings.

Average the six input embeddings

First attempt: let every source send its original embedding into the update.

input

1 · She

2 · deposited

3 · money

embedding

$$e_1 = (0 \ 0 \ 1)$$

$$e_2 = (1 \ 0 \ 1)$$

$$e_3 = (2 \ 0 \ 0)$$

input

4 · in

5 · the

6 · bank

embedding

$$e_4 = (0 \ 1 \ 0)$$

$$e_5 = (0 \ 0 \ 0)$$

$$e_6 = (1 \ 1 \ 0)$$

$$\Delta e_6 = \frac{1}{6} \sum_{j=1}^6 e_j = \frac{1}{6} (4 \ 2 \ 2) = \left(\frac{2}{3} \ \frac{1}{3} \ \frac{1}{3} \right)$$

| We would add this vector to e_6 . But can an equal average select the useful clues?

Hand-chosen numbers. We will reuse this same embedding table in the complete model.

Problem 1: equal weights cannot select a clue

She deposited money in the **bank** _ _ _

0.167

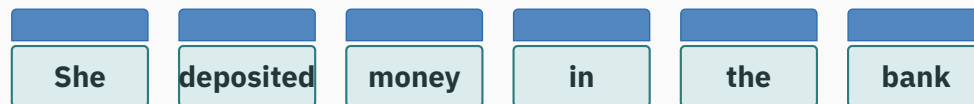
0.167

0.167

0.167

0.167

0.167



| The mean gives money and the exactly the same multiplier: $\frac{1}{6}$.

| If finance-related information is useful here, we want a way to give its sources more weight.

Problem 2: an average forgets order

WHO TOLD WHOM?

Maya told Ravi about the bank

WHO TOLD WHOM?

Ravi told Maya about the bank

same tokens $\rightarrow$ same mean same final token $\rightarrow$ same e_{last}

Same average update and the same final bank embedding—but different people are speaking.

We will add position information before the parallel model. First solve Problem 1: deciding which sources contribute more.

Give useful sources more weight

She deposited money in the bank ____

$$\Delta e_6 = \sum_{j=1}^6 \alpha_j e_j$$

$$\alpha_j \geq 0, \quad \sum_j \alpha_j = 1$$

THE OLD MEAN

All six multipliers are $\frac{1}{6}$.

A WEIGHTED UPDATE

The multiplier for money can differ from the multiplier for the.

For now, sources send their embeddings unchanged. The weights decide how much each source contributes. How should we choose them?

The receiving position needs a say

She deposited money in the bank ___

- **Distance alone?** The nearest word is the. Nearby does not necessarily mean informative.
- **Fixed importance per word?** Always preferring money cannot express every position's information needs.
- **Match the current need.** Use the representation at bank to form a request. Compare it with each source's matching features.

| Weights should depend on both the receiving position and the source.

What information would help after bank?

She deposited money in the **bank** ____

bank is known input 6. We want to predict the token **after it**.

First, which clues could help enrich bank's embedding?

Possible information to seek

Is there finance-related information?

What action happened?

Who is involved?

Clue in prefix

money

deposited

She

A **query** encodes a request as numbers. For our toy calculation, let q_6 seek finance-related clues.

These questions explain possible roles. The model computes a vector, not an English question.

Turn bank's embedding into a query

The query comes from the **known bank embedding**, not from the blank.

$$q_6 = \underbrace{\begin{pmatrix} 1 & 1 & 0 \end{pmatrix}}_{e_6 \quad (1 \times 3)} \underbrace{\begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}}_{W_Q \quad (3 \times 3)}$$

Our toy W_Q is shared and trainable. It keeps coordinate 1 only because of the starting entries we chose.

coordinate 1 $1 \times 1 + 1 \times 0 + 0 \times 0 = 1$
coordinate 2 $1 \times 0 + 1 \times 0 + 0 \times 0 = 0$
coordinate 3 $1 \times 0 + 1 \times 0 + 0 \times 0 = 0$

$$q_6 = \begin{pmatrix} 1 & 0 & 0 \end{pmatrix} \quad \text{query shape: } 1 \times 3$$

PyTorch

```
e6 = torch.tensor([[1., 1., 0.]])          # [1, 3]
W_Q = torch.diag(torch.tensor([1., 0., 0.]))
q6 = e6 @ W_Q                             # [[1., 0., 0.]]
```

Row-by-column teaching approach: Visual AI, 3:19. Bank numbers are our toy example.

A key describes what a source can match

The query describes the request. Each source supplies a **key** to match against it.

$$k_j = e_j W_K$$

For this toy, $W_K = I$: the identity map leaves all three coordinates unchanged.

source

3 · money

5 · the

6 · bank

lookup embedding

$$e_3 = \begin{pmatrix} 2 & 0 & 0 \end{pmatrix}$$

$$e_5 = \begin{pmatrix} 0 & 0 & 0 \end{pmatrix}$$

$$e_6 = \begin{pmatrix} 1 & 1 & 0 \end{pmatrix}$$

matching key

$$k_3 = \begin{pmatrix} 2 & 0 & 0 \end{pmatrix}$$

$$k_5 = \begin{pmatrix} 0 & 0 & 0 \end{pmatrix}$$

$$k_6 = \begin{pmatrix} 1 & 1 & 0 \end{pmatrix}$$

For intuition, call matching coordinate 1 a **finance-related cue**. Our query $(1, 0, 0)$ reads this coordinate and ignores the other two.

Hand-chosen features, not discovered meanings. W_K is trainable; identity is only our starting choice.

Match the request, not another copy of bank

$q_6 = (1 \ 0 \ 0)$ seek the first matching cue

Three highlighted sources for now. The later softmax will use **all six**.

source	key	$q_6 \cdot k_j$	score
3 · money	$(2 \ 0 \ 0)$	$1 \times 2 + 0 \times 0 + 0 \times 0$	2
5 · the	$(0 \ 0 \ 0)$	$1 \times 0 + 0 \times 0 + 0 \times 0$	0
6 · bank	$(1 \ 1 \ 0)$	$1 \times 1 + 0 \times 1 + 0 \times 0$	1

money matches the requested direction more strongly than bank's own key. We compare q_6 with k_6 —not with itself.

These are match scores, not attention weights or next-token probabilities.

Unscaled dot products for now: a three-coordinate query and key give one score.

The same query can find different evidence

She **deposited money** in the bank ___

She **watched water** by the bank ___

Same word at position 6. Does the query already know which meaning applies?

$$\underbrace{e_6 = (1 \ 1 \ 0)}_{\text{same lookup}} \xrightarrow{\text{same } W_Q} \underbrace{q_6 = (1 \ 0 \ 0)}_{\text{same query}}$$

Not at this first step: q_6 comes from bank's lookup embedding. It does **not** yet summarize the whole prefix.

The source embeddings differ, so their keys can differ. The same query can find different matches in the two prefixes.

Matching decides **where** to get information. Next: what should each source **send**?

Qualitative comparison only. We have not trained this toy model to distinguish both meanings.

Value: what should this source send?

Earlier, a source sent its embedding unchanged. Now learn **what it should send**.

$e_6 = (1 \ 1 \ 0)$ the lookup embedding for bank

KEY: HELP CHOOSE AN AMOUNT

$$k_6 = e_6 W_K = (1 \ 1 \ 0)$$

Compare this with a query.

VALUE: SUPPLY THE INFORMATION

$$v_6 = e_6 W_V = (1 \ 2 \ 0)$$

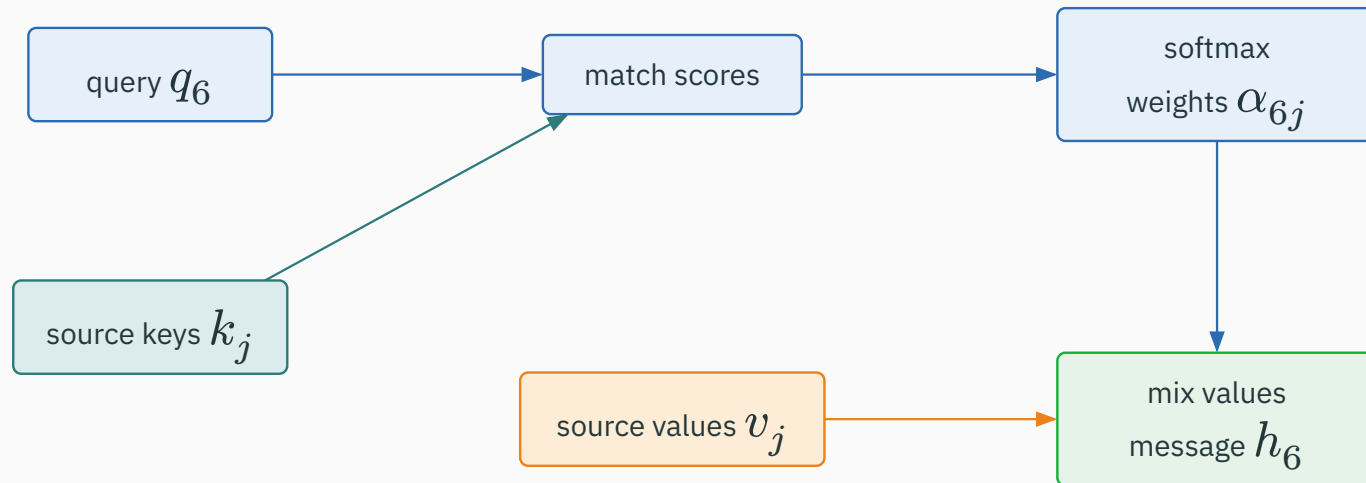
Information to add at the receiver.

e_j represents the source token. $v_j = e_j W_V$ is the information it sends—not its finished context-enriched embedding.

Here $W_K = I$ and $W_V = \text{diag}(1, 2, 1)$. If $W_V = I$, sending $v_j = e_j$ is a valid special case.

Match queries and keys; mix the values

She deposited money in the **bank** ---



The weight computed using k_3 multiplies v_3 : the key and value come from the **same source position**.

α_{6j} : receiver 6, source j . The same source embedding creates its key and value.

Turn the collected message into an embedding update

She deposited money in the **bank** ---

$$h_6 = \sum_{j=1}^6 \alpha_{6j} v_j$$

h_6 : the message gathered for bank using its query—not a new token.

For now, use this message directly as the update: $\Delta e_6 = h_6$. Later we will learn a map from message to update.

$$\begin{array}{ccccc} e_6 & & \Delta e_6 & & e'_6 \\ \text{lookup embedding} & + & \text{update from context} & = & \text{context-enriched} \\ & & & & \text{embedding} \end{array}$$

Information from money and deposited can now enter e'_6 . The predictor reads this enriched embedding.

Same matches can carry different messages

A controlled experiment: keep all inputs, queries and keys fixed.

ORIGINAL VALUE TRANSFORMATION

Scale coordinates by $(1, 2, 1)$.

money sends $v_3 = (2 \ 0 \ 0)$.

STOP SENDING COORDINATE 1

Scale coordinates by $(0, 2, 1)$.

money now sends $\tilde{v}_3 = (0 \ 0 \ 0)$.

Do the matches change? Does the message change?

Scores and weights stay the same. Every value's first coordinate becomes zero, so the message's first coordinate becomes zero too.

The original e_6 still reaches the addition. Suppressing incoming information does not erase what bank already carries.

This deliberately changes W_V only. It is not a word replacement or an optimizer step. It is verified in the runnable companion.

A contextual update is not a learning update

WHILE READING THIS PREFIX

$$e_6 \rightarrow e'_6$$

Compute a new representation for **this occurrence** of bank.

The stored table E and all matrices stay unchanged.

WHILE TRAINING THE MODEL

$$E, W_Q, W_K, W_V, \dots$$

A loss guides updates to shared **parameters**.

The next forward pass recomputes the intermediate vectors.

The two bank prefixes can start with the same lookup row—and even the same first-layer query. Their different source vectors can produce different messages.

Our bank model will predict after adding context

She deposited money in the bank ___

$$e'_6 = e_6 + \Delta e_6$$

$$\Delta e_6 = h_6 W_O$$

Start with $W_O = I$ and three coordinates throughout: the message itself is the update.
Later, learn how to write it back.

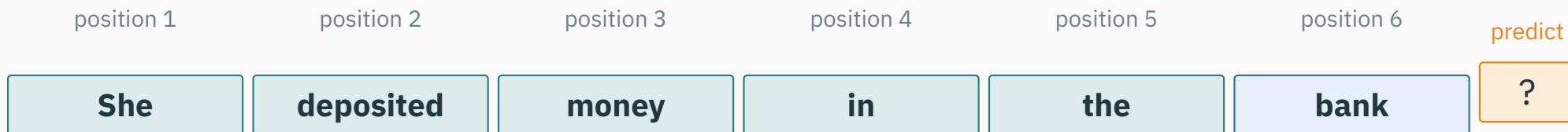
$e'_6 \rightarrow$ vocabulary scores $\rightarrow$ next-token probabilities

Now put numbers on every step: query, keys, values, weights, message, addition, and prediction.

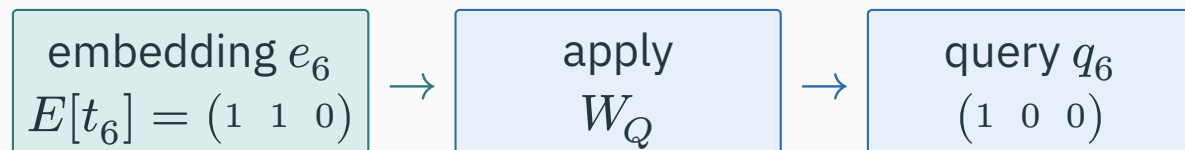
Return to the original value transformation (scale by 1, 2, 1) for the calculation below.
This small residual-attention model will become a full Transformer block in Lecture 3.

Keep the query; now inspect all six sources

One word per token. Position i tells us where; token ID t_i tells us which word.



All six words are known. The missing next token is not an input.



$q_6 = e_6 W_Q$. We use lookup embeddings now; position information comes later.

Keep the same query. Now include every known input position in the calculation.



Give each input a key and a value

Last known token: bank, position 6. $q_6 = (1 \ 0 \ 0)$

$k_j = e_j W_K, v_j = e_j W_V$. The query, keys, and values each have three coordinates here.

position	known word	key: used to match	value: information to mix
1	She	$k_1 = (0 \ 0 \ 1)$	$v_1 = (0 \ 0 \ 1)$
2	deposited	$k_2 = (1 \ 0 \ 1)$	$v_2 = (1 \ 0 \ 1)$
3	money	$k_3 = (2 \ 0 \ 0)$	$v_3 = (2 \ 0 \ 0)$
4	in	$k_4 = (0 \ 1 \ 0)$	$v_4 = (0 \ 2 \ 0)$
5	the	$k_5 = (0 \ 0 \ 0)$	$v_5 = (0 \ 0 \ 0)$
6	bank	$k_6 = (1 \ 1 \ 0)$	$v_6 = (1 \ 2 \ 0)$

Compare the query with a key to choose a weight. Use that weight to multiply the value from the same position.

Made-up numbers for this calculation. Real learned coordinates need not have simple meanings.

Compare the same query with all six keys

Last known token: bank, position 6. $q_6 = (1 \ 0 \ 0)$

input position	key k_j	multiply the coordinates, then add	score s_j
1 · She	$(0 \ 0 \ 1)$	$1 \times 0 + 0 \times 0 + 0 \times 1$	0
2 · deposited	$(1 \ 0 \ 1)$	$1 \times 1 + 0 \times 0 + 0 \times 1$	1
3 · money	$(2 \ 0 \ 0)$	$1 \times 2 + 0 \times 0 + 0 \times 0$	2
4 · in	$(0 \ 1 \ 0)$	$1 \times 0 + 0 \times 1 + 0 \times 0$	0
5 · the	$(0 \ 0 \ 0)$	$1 \times 0 + 0 \times 0 + 0 \times 0$	0
6 · bank	$(1 \ 1 \ 0)$	$1 \times 1 + 0 \times 1 + 0 \times 0$	1

At bank: $q_6 = (1 \ 0 \ 0)$, but $k_6 = (1 \ 1 \ 0)$. Different vectors, different jobs.

Here, money matches this query better than bank's own key.

Unscaled dot products for now. We will add the usual scaling factor later.

Exponentiate the six match scores

The query at **bank** gave scores $s = (0 \ 1 \ 2 \ 0 \ 0 \ 1)$.

input position	score s_j	exponentiate	positive amount
1 · She	0	$\exp(0)$	1.000
2 · deposited	1	$\exp(1)$	2.718
3 · money	2	$\exp(2)$	7.389
4 · in	0	$\exp(0)$	1.000
5 · the	0	$\exp(0)$	1.000
6 · bank	1	$\exp(1)$	2.718

$$D \approx 1.000 + 2.718 + 7.389 + 1.000 + 1.000 + 2.718 = 15.825$$

| Larger scores give larger positive amounts. These are not weights yet: their total is 15.825, not 1.

For this hand calculation, we round the exponentials first, then add the displayed values.

Divide every amount by the same total

Same six sources. Same total: $D \approx 15.825$.

In α_{6j} , 6 is the receiver and j the source. Here α_j abbreviates α_{6j} .

input position	amount		total		weight α_j
1 · She	1.000	÷	15.825	≈	0.063
2 · deposited	2.718	÷	15.825	≈	0.172
3 · money	7.389	÷	15.825	≈	0.467
4 · in	1.000	÷	15.825	≈	0.063
5 · the	1.000	÷	15.825	≈	0.063
6 · bank	2.718	÷	15.825	≈	0.172

$$\alpha_j = \frac{\exp(s_j)}{\sum_{m=1}^6 \exp(s_m)}$$

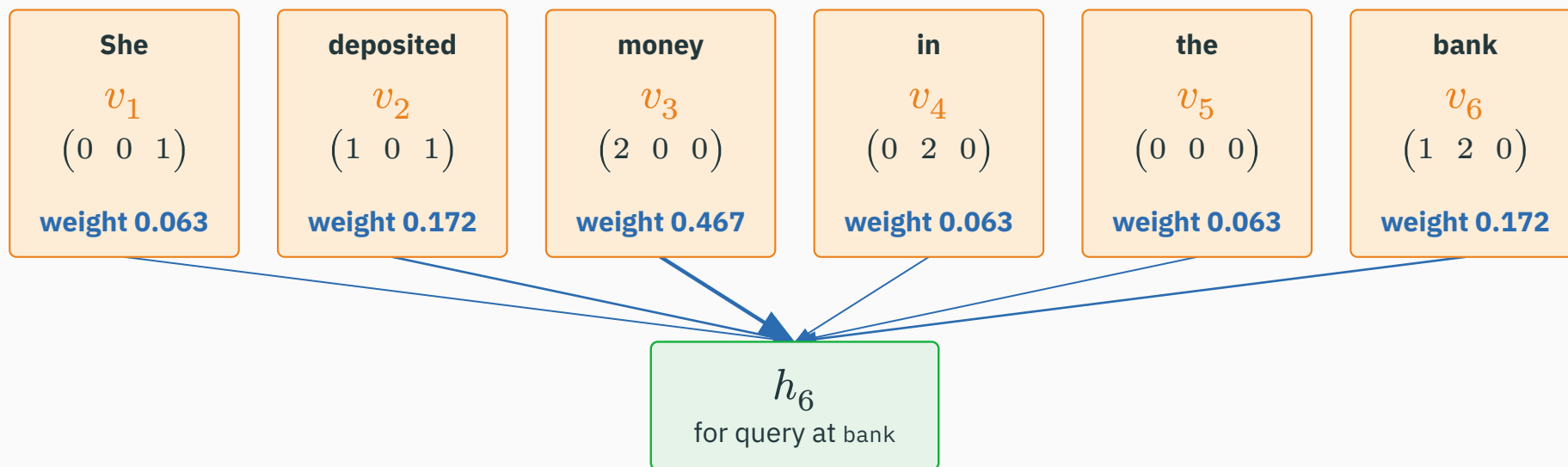
Exponentiate, then divide by the total: this is **softmax**.

Six source positions give six weights; the displayed weights sum to 1.

We will reuse these three-decimal weights when mixing the values.

Use each weight to mix the corresponding value

Last known token: bank, position 6. $q_6 = (1 \ 0 \ 0)$



$$h_6 = 0.063v_1 + 0.172v_2 + 0.467v_3 + 0.063v_4 + 0.063v_5 + 0.172v_6$$

| Multiply every coordinate of a value by its weight. Then add the six vectors.

Calculate the three coordinates of the message

$h_{6,c}$ is coordinate c of the message for receiver 6. Use the same weights for each coordinate.

First coordinate

$$h_{6,1} = 0.063 \times 0 + 0.172 \times 1 + 0.467 \times 2 + 0.063 \times 0 + 0.063 \times 0 + 0.172 \times 1 = 1.278$$

Second coordinate

$$h_{6,2} = 0.063 \times 0 + 0.172 \times 0 + 0.467 \times 0 + 0.063 \times 2 + 0.063 \times 0 + 0.172 \times 2 = 0.470$$

Third coordinate

$$h_{6,3} = 0.063 \times 1 + 0.172 \times 1 + 0.467 \times 0 + 0.063 \times 0 + 0.063 \times 0 + 0.172 \times 0 = 0.235$$

$$h_6 \approx (1.278 \ 0.470 \ 0.235)$$

Six weights produced a three-coordinate message for this occurrence of bank.

Keep bank, then add what the context contributes

She deposited money in the bank _ _ _

LOOKUP EMBEDDING AT BANK

$$e_6 = (1 \ 1 \ 0)$$

INCOMING MESSAGE

$$h_6 \approx (1.278 \ 0.470 \ 0.235)$$

Keep the unscaled hand calculation for this first addition.

$$\Delta e_6 = h_6 \quad \text{for now: add the message unchanged}$$

$$e'_6 = e_6 + \Delta e_6 \approx (2.278 \ 1.470 \ 0.235)$$

Predict the next token from e'_6 : bank's embedding **plus** contextual information. If the message is zero, e_6 remains.

This changes the representation for this input. It does not overwrite bank's row in E .

Start with one embedding row per token

E : eight token types $\times$ three coordinates. IDs run from 0 to 7; input positions from 1 to 6.

ID	token	embedding row
0	She	$\begin{pmatrix} 0 & 0 & 1 \end{pmatrix}$
1	deposited	$\begin{pmatrix} 1 & 0 & 1 \end{pmatrix}$
2	money	$\begin{pmatrix} 2 & 0 & 0 \end{pmatrix}$
3	in	$\begin{pmatrix} 0 & 1 & 0 \end{pmatrix}$
4	the	$\begin{pmatrix} 0 & 0 & 0 \end{pmatrix}$
5	bank	$\begin{pmatrix} 1 & 1 & 0 \end{pmatrix}$
6	period (.)	$\begin{pmatrix} 0 & 0 & 0.5 \end{pmatrix}$
7	and	$\begin{pmatrix} 0.5 & 1 & 1 \end{pmatrix}$

$$X = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 1 \\ 2 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \\ 1 & 1 & 0 \end{pmatrix}$$

$e_i = E[t_i]$. Row i of X is e_i .

X : six sequence rows, 6×3 .

PyTorch

```
X = E[ids]           # rows e_i, [6, 3]
e6 = X[-1]           # bank at position 6
```

Here $t_6 = 5$: position 6 looks up token ID 5, the row for bank.

Three learned matrices give one vector three jobs

$e_6 = (1 \ 1 \ 0)$ the lookup embedding at bank

transform

$e_6 W_Q$
 $e_6 W_K$
 $e_6 W_V$

result

q_6
 k_6
 v_6

job in this calculation

What context would help continue after bank?
When should another query match this source?
What information should this source contribute?

Each W is a shared, learned transformation. Every input position uses the same three matrices. Their outputs are new vectors, not new parameters.

$W_Q, W_K \in \mathbb{R}^{d \times d_k}; W_V \in \mathbb{R}^{d \times d_v}$. Here $d = d_k = d_v = 3$.

These questions describe roles; the model computes vectors, not English questions.

Use the same query map at every position

$$q_i = e_i W_Q \quad W_Q = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

One shared 3×3 matrix. It is not a separate learned matrix for each word.

position

1 · She
3 · money
6 · bank

embedding row

$$e_1 = (0 \ 0 \ 1) \\ e_3 = (2 \ 0 \ 0) \\ e_6 = (1 \ 1 \ 0)$$

query row

$$q_1 = (0 \ 0 \ 0) \\ q_3 = (2 \ 0 \ 0) \\ q_6 = (1 \ 0 \ 0)$$

$$\underbrace{Q}_{6 \times 3} = \underbrace{X}_{6 \times 3} \underbrace{W_Q}_{3 \times 3}$$

PyTorch

```
Q = X @ W_Q          # [6, 3]
q6 = Q[-1]            # [3]
```

Q stacks all six queries. We use its bank row to predict after this prefix.

In math queries are rows; PyTorch's Q[-1] is a one-dimensional tensor.

The other two projections reproduce our keys and values

$$W_K = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad W_V = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$K = XW_K = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 1 \\ 2 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \\ 1 & 1 & 0 \end{pmatrix}$$

$$V = XW_V = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 1 \\ 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \\ 1 & 2 & 0 \end{pmatrix}$$

Rows in both matrices: She · deposited · money · in · the · bank. Both are 6×3 .

PyTorch

```
K = X @ W_K          # matching vectors
V = X @ W_V          # information to mix
```

These simple initial matrices reproduce every vector in our hand calculation. Training can change every entry.

Is the value just the original embedding?

LOOKUP EMBEDDING

$$e_6 = (1 \ 1 \ 0)$$

The row read from the table E .

VALUE SUPPLIED TO THE MIXTURE

$$v_6 = e_6 W_V = (1 \ 2 \ 0)$$

A learned transformation of that row.

If $W_V = I$, then $v_j = e_j$: using the original embedding as a value is valid. Learning W_V lets the model learn **what information to send**.

A learned W_V selects, combines, or reweights input features. Queries and keys choose the mixing amounts; values supply the information to mix.

Here all widths are 3. Query/key widths must agree; value width may differ. Coordinates need not have simple meanings.

Map the incoming message into the space we are updating

$$h_6 \in \mathbb{R}^{1 \times d_v} \xrightarrow{W_O} \Delta e_6 \in \mathbb{R}^{1 \times d}$$

So far we passed the message through unchanged. A learned W_O can translate it into the embedding coordinates before addition.

$$W_O \in \mathbb{R}^{d_v \times d}, \quad e'_6 = e_6 + h_6 W_O$$

Here $d_v = d = 3$ and we initialize $W_O = I$. For now the message passes through unchanged. Training can change that map too.

PyTorch

```
delta6 = h6 @ W_0          # message in embedding coordinates
e6_new = X[-1] + delta6     # retain bank + add context
```

Why divide a dot product by the square root of its width?

$$s_j = \frac{q_6 \cdot k_j}{\sqrt{d_k}}, \quad d_k = 3$$

The dot product adds d_k products. With more coordinates, its typical magnitude grows even without a better match.

query/key width

$$d_k = 4$$
$$d_k = 64$$

typical score scale

$$\sqrt{4} = 2$$
$$\sqrt{64} = 8$$

after division

$$2 \div 2 = 1$$
$$8 \div 8 = 1$$

Under the simple assumption of independent, zero-mean, unit-variance coordinates: variance of the dot product is d_k .

Scaling keeps softmax from becoming too sharp just because vectors are wider. It does not normalize each vector or force a self-match to win.

Vaswani et al., §3.2.1. Standard deviation, not variance, gives the square root.

Use scaled scores for the rest of the model

$$\sqrt{3} \approx 1.732 \quad s_j = \frac{q_6 \cdot k_j}{\sqrt{3}}$$

input	dot product	scaled score	weight
She	0	0.000	0.103
deposited	1	0.577	0.183
money	2	1.155	0.326
in	0	0.000	0.103
the	0	0.000	0.103
bank	1	0.577	0.183

$$h_6 = \sum_j \alpha_j v_j \approx (1.018 \ 0.571 \ 0.286)$$

Scaled numbers replace the warm-up; displayed weights are rounded. $\alpha_j = \alpha_{6j}$: source j to bank.

PyTorch

```
s = (Q[-1] @ K.T) / math.sqrt(K.shape[-1])
a = s.softmax(dim=-1)
h6 = a @ V          # use full-precision weights
```

Predict from bank plus its contextual update

$$e'_6 = e_6 + h_6 W_O \approx (2.018 \ 1.571 \ 0.286)$$

$$z_6 = e'_6 U + b \quad (1 \times 3)(3 \times 8) + (1 \times 8)$$

Vocabulary column order: She · deposited · money · in · the · bank · period (.) · and

$$U = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}, \quad b = 0_{1 \times 8}$$

$$z_{6,\text{period}} = 2.018(1) + 1.571(0) + 0.286(0) \approx 2.018$$

$z_{6,\text{and}} \approx 1.571$; the other six logits are 0.

PyTorch

```
e6_new = X[-1] + h6 @ W_O    # [3]
logits = e6_new @ U + b      # [8]
```

Vocabulary softmax answers “what could come next?”

She deposited money in the bank ___

candidate token

She, deposited, money, in, the,
bank
period (.)
and

logit

0.000 each
2.018
1.571

probability

0.055 each
0.410
0.263

$$p(.) = \frac{\exp(2.018)}{6 \exp(0) + \exp(2.018) + \exp(1.571)} \approx 0.410$$

The calculation uses unrounded logits. The six zero logits still receive probability mass.

Probabilities are rounded for display, so the displayed entries need not sum exactly to 1.

Attention softmax: **six source positions**, used to mix vectors.

Vocabulary softmax: **eight token types**, used to choose the next token.

Follow one complete forward pass

She deposited money in the bank ___

operation

look up embeddings
make matching vectors
make value vectors
score, scale, softmax
mix values
map and add the message
predict next token

tensor / shape

$X : 6 \times 3$
 $Q, K : 6 \times 3$
 $V : 6 \times 3$
 $s, \alpha : 1 \times 6$
 $h_6 : 1 \times 3$
 $e'_6 : 1 \times 3$
 $z_6, p : 1 \times 8$

one row we followed

$e_6 = (1 \ 1 \ 0)$
 $q_6 = (1 \ 0 \ 0)$
 $v_6 = (1 \ 2 \ 0)$
money weight ≈ 0.326
 $(1.018 \ 0.571 \ 0.286)$
 $(2.018 \ 1.571 \ 0.286)$
 $p(.) \approx 0.410$

X stacks e_i ; ΔX stacks Δe_i ; X' stacks e'_i . Capital E remains the lookup table.

In the next snippets, `forward_last(ids)` runs this calculation and returns the last row's logits. Parameters stay fixed.

Generate by choosing one token from this distribution

She deposited money in the bank ___

$$p(.) \approx 0.410, \quad p(\text{and}) \approx 0.263$$

Greedy decoding chooses the period token here. Sampling can choose another token with nonzero probability. Suppose this sample is and.

She deposited money in the bank and ___

PyTorch

```
p = logits.softmax(dim=-1) # probabilities over vocabulary
next_id = torch.multinomial(p, num_samples=1)
ids = torch.cat([ids, next_id])
```

“and” is one possible sample, not the most probable token or the observed training answer.

The appended token supplies the next query

She deposited money in the bank and ___

generation step

after bank, position 6
after and, position 7

last input vector

$$e_6 = \begin{pmatrix} 1 & 1 & 0 \end{pmatrix}$$
$$e_7 = \begin{pmatrix} 0.5 & 1 & 1 \end{pmatrix}$$

last query

$$q_6 = \begin{pmatrix} 1 & 0 & 0 \end{pmatrix}$$
$$q_7 = \begin{pmatrix} 0.5 & 0 & 0 \end{pmatrix}$$

The new token gets its own lookup embedding e_7 , not bank's e'_6 . Its query collects h_7 from seven sources; add the mapped message to e_7 .

PyTorch

```
X = E[ids] # now [7, 3]
Q, K, V = X @ W_Q, X @ W_K, X @ W_V
a = ((Q[-1] @ K.T) / math.sqrt(3)).softmax(-1)
h7 = a @ V
```

The matrices are unchanged. The last input and the set of available sources changed.

Reuse the same model for every generation step

$\text{prefix} \rightarrow h_{\text{last}} \rightarrow e'_{\text{last}} \rightarrow p(\text{next}) \rightarrow \text{append}$

$$h_7 \approx (0.791 \ 0.811 \ 0.406)$$

$$e'_7 = e_7 + h_7 W_O \approx (1.291 \ 1.811 \ 1.406)$$

After the possible sample and: $p(\cdot) \approx 0.231$ and $p(\text{and}) \approx 0.388$.

PyTorch

```
p = forward_last(ids).softmax(-1)
next_id = torch.multinomial(p, 1)
ids = torch.cat([ids, next_id])
```

Generation changes the inputs, with **fixed parameters**. Training uses observed targets to change the parameters.

A repeated last word can have the same first-layer query in this content-only toy. Growing context can still change the mixture.

The observed next token tells us the prediction error

Training text: She deposited money in the bank .

Return to the six-token prefix, before the sampled and. The target is the period token (.). It was never an input to this prediction.

$$L = -\ln p(. \mid \text{She deposited money in the bank}) \approx -\ln(0.410) \approx 0.891$$

This is one next-token cross-entropy loss, using the natural logarithm.

PyTorch

```
ids = torch.tensor([0, 1, 2, 3, 4, 5])
logits = forward_last(ids)
target = torch.tensor(6)      # vocabulary ID for '.'
loss = F.cross_entropy(logits, target)
```

Pass logits to cross_entropy. It applies the required log-softmax internally.

Which numbers will the optimizer change?

LEARNED PARAMETERS

E : token embedding table

W_Q, W_K, W_V, W_O : projection matrices

U, b : vocabulary predictor

Stored across examples. Updated by the optimizer.

RECOMPUTED INTERMEDIATES

e_i, Q, K, V : this input's vectors

s, α : scores and attention weights

$h, \Delta e, e'$: message and contextual embedding

z, p, L : prediction and loss

Recomputed from the inputs and current parameters.

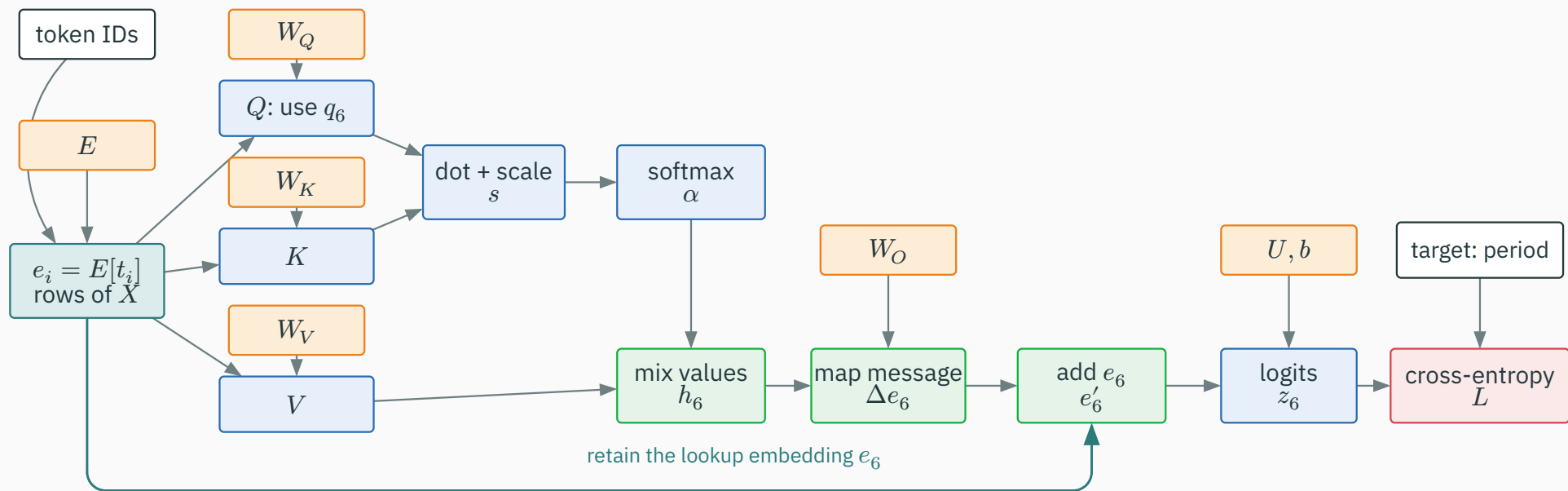
Adding context gives $e'_6 = e_6 + \Delta e_6$ for this input. Learning changes the shared parameters, including E . These are two different kinds of update.

PyTorch

```
params = [E, W_Q, W_K, W_V, W_O, U, b]  
optimizer = torch.optim.SGD(params, lr=0.1)
```

The forward pass is a computational graph

Input: She deposited money in the bank. Orange boxes are learned parameters.

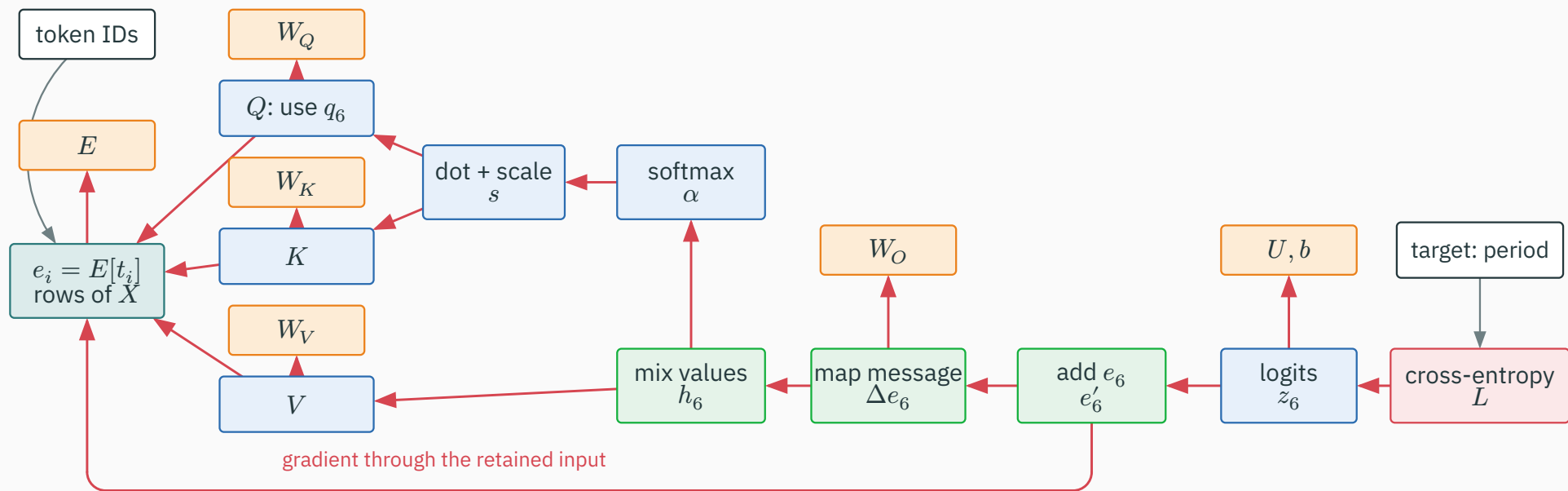


Follow the arrows to the loss. Each operation records which tensors produced its output.

The retained input and incoming message meet at addition. The updated state supplies logits for the observed period target.

Autograd follows the graph backward

Red arrows carry loss gradients back through the same operations.



PyTorch

```
loss.backward()
grad_WQ = W_Q.grad      # dL / dW_Q
grad_E = E.grad         # dL / dE
```

Gradients flow through matching, values, and the retained-input path. Learned tensors require gradients; integer IDs and targets are data.

Apply the update to all learned parameters

$$\theta = \{E, W_Q, W_K, W_V, W_O, U, b\}, \quad \theta \leftarrow \theta - \eta \nabla_{\theta} L$$

Autograd computes the gradients. The optimizer uses them to change the learned parameters. We do not need to derive each gradient by hand.

PyTorch

```
optimizer.zero_grad()
loss = F.cross_entropy(forward_last(ids), target)
loss.backward()
optimizer.step()
```

Use the original six-token prefix and learning rate $\eta = 0.1$. The next forward pass recomputes the queries, keys, values, and prediction from the updated parameters.

Run the same prefix again after the update

She deposited money in the bank _ _ _

computed quantity

lookup e_6 from table E

incoming message h_6

mapped message Δe_6

contextual embedding e'_6

probability of the period token

loss $-\ln p(\cdot)$

before SGD

(1.000 1.000 0.000)

(1.018 0.571 0.286)

(1.018 0.571 0.286)

(2.018 1.571 0.286)

0.410

0.891

after SGD

(1.091 0.964 0.000)

(1.196 0.488 0.272)

(1.289 0.446 0.272)

(2.379 1.410 0.272)

0.658

0.419

| Learning changed E , so the lookup e_6 changed too. W_O is no longer identity: h_6 and Δe_6 now differ.

| The model is still tiny and barely trained. Learning on one example does not establish good predictions on unseen text.

Displayed rows are rounded separately; additions use full precision. Next, train at every position.

Predict every next token without reading ahead

Complete sentence: She deposited money in the bank .

input	She	deposited	money	in	the	bank
target	deposited	money	in	the	bank	.

Input position i predicts the target $y_i = t_{i+1}$.

At money (position 3), the target is in (position 4).

query 3	She	deposited	money	in	the	bank
---------	-----	-----------	-------	----	-----	------

Read She deposited money. Hide in the bank from this query.

$$M_{3,:} = (0 \ 0 \ 0 \ -\infty \ -\infty \ -\infty)$$

Add the mask before softmax. Positions 4, 5, and 6 then get zero attention weight in row 3.

Give each position its own vector

$$e_i^{(0)} = e_i + r_i \quad \text{and} \quad e_i^{(1)} = e_i^{(0)} + \Delta e_i$$

Superscripts count stages: (0) before attention, (1) after. The subscript i is still the position, not the token ID.

LOOKUP EMBEDDING

POSITION 6

STAGE 0 AT POSITION 6

$$e_6 = E[t_6] \\ \begin{pmatrix} 1 & 1 & 0 \end{pmatrix}$$

+

$$r_6 \\ \begin{pmatrix} 0 & 0.1 & 0 \end{pmatrix}$$

→

$$e_6^{(0)} \\ \begin{pmatrix} 1 & 1.1 & 0 \end{pmatrix}$$

$$e_6^{(0)} W_Q \rightarrow q_6 \quad e_6^{(0)} W_K \rightarrow k_6 \quad e_6^{(0)} W_V \rightarrow v_6$$

Row i of X is now $e_i^{(0)}$. The projections decide how position affects Q, K, and V.

R : max-length $\times d$ position table; $r_i = R[i - 1]$ in code. Restart original parameters with $R = 0$; the nonzero r_6 above only illustrates addition.

Attention can use word-position pairs. Taking their plain mean still loses order.

Compute all six query rows together

QUERIES

$$Q = XW_Q \in \mathbb{R}^{6 \times 3}$$

The same projections act on every row $e_i^{(0)}$ of X . X' will stack the output rows $e_i^{(1)}$.

$$S = Q \frac{K^\top}{\sqrt{3}} \in \mathbb{R}^{6 \times 6}$$

$$A = \text{softmax}(S + M) \in \mathbb{R}^{6 \times 6}$$

$$A_{ij} = \alpha_{ij}: \text{receiver } i, \text{ source } j.$$

Softmax runs across each row. Earlier $\alpha_j = \alpha_{6j}$.

KEYS

$$K = XW_K \in \mathbb{R}^{6 \times 3}$$

VALUES

$$V = XW_V \in \mathbb{R}^{6 \times 3}$$

CAUSAL MASK M

	1	2	3	4	5	6
1	0	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$
2	0	0	$-\infty$	$-\infty$	$-\infty$	$-\infty$
3	0	0	0	$-\infty$	$-\infty$	$-\infty$
4	0	0	0	0	$-\infty$	$-\infty$
5	0	0	0	0	0	$-\infty$
6	0	0	0	0	0	0

$$H = AV \quad \Delta X = HW_O \quad X' = X + \Delta X$$

$$Z = X'U + b \in \mathbb{R}^{6 \times 8}$$

ΔX stacks Δe_i ; X' stacks $e_i^{(1)} = e_i^{(0)} + \Delta e_i$. All three matrices $H, \Delta X, X'$ are 6×3 .

Row 3 predicts in using positions 1, 2, and 3. All six rows run in the same forward pass.

The matrix pass contains our earlier calculation

1 = She 2 = deposited 3 = money 4 = in 5 = the 6 = bank

A : weights over source positions

$H = AV$: one message per row

	1	2	3	4	5	6
1	1.000	0.000	0.000	0.000	0.000	0.000
2	0.360	0.640	0.000	0.000	0.000	0.000
3	0.070	0.223	0.707	0.000	0.000	0.000
4	0.250	0.250	0.250	0.250	0.000	0.000
5	0.200	0.200	0.200	0.200	0.200	0.000
6	0.103	0.183	0.326	0.103	0.103	0.183



	coord. 1	coord. 2	coord. 3
1	0.000	0.000	1.000
2	0.640	0.000	1.000
3	1.637	0.000	0.293
4	0.750	0.500	0.500
5	0.600	0.400	0.400
6	1.018	0.571	0.286

rows: query positions columns: source positions

Three coordinates per query position.

Original parameters, $R = 0$. Round only for display: compute H with full-precision weights.

Row 3 cannot read positions 4, 5, or 6: those weights are zero.

With $R = 0$, $e_6^{(0)} = e_6$. The same $h_6 \approx (1.018 \ 0.571 \ 0.286)$ gives $e_6^{(1)} = e_6^{(0)} + \Delta e_6 \approx (2.018 \ 1.571 \ 0.286)$.

The complete forward pass in PyTorch

Original parameters and $R = 0$. Rows of X are $e_i^{(0)}$; rows of X_{new} are $e_i^{(1)}$.

PyTorch

```
def forward(ids):  
    X = E[ids] + R[torch.arange(len(ids))]  
    Q, K, V = X @ W_Q, X @ W_K, X @ W_V
```

PyTorch

```
    scores = Q @ K.T / Q.shape[-1] ** 0.5  
    future = torch.ones_like(scores, dtype=torch.bool).triu(1)  
    A = scores.masked_fill(future, -torch.inf).softmax(-1)
```

PyTorch

```
    H = A @ V  
    X_new = X + H @ W_O  
    return X_new @ U + b  
all_logits = forward(ids)  # [6, 8]
```

Average the six losses, then update the parameters

Restart from the original toy parameters, with $R = 0$. Targets: deposited money in the bank .

$$L = -\frac{1}{6} \sum_{i=1}^6 \log \text{softmax}(Z_{i,:})_{y_i}$$

Before the update: mean loss ≈ 2.339914 .

PyTorch

```
target_ids = torch.tensor([1, 2, 3, 4, 5, 6])
params = [E, R, W_Q, W_K, W_V, W_O, U, b]
optimizer = torch.optim.SGD(params, lr=0.1)
```

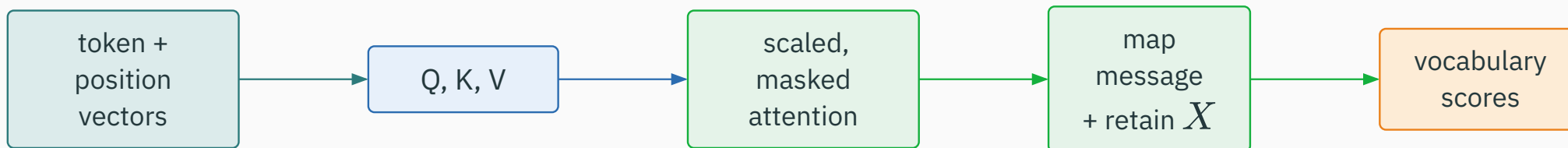
PyTorch

```
optimizer.zero_grad()
loss = F.cross_entropy(all_logits, target_ids)
loss.backward()
optimizer.step()
```

Mean loss: 2.339914 → 2.199261

The last row's loss rises: 0.891023 to 0.947535. The average over all six rows falls.

One complete next-token model



TRAINING

Known sentence

- six next-token losses
- one parameter update

GENERATION

Current prefix

- next-token probabilities
- choose a token and append it

Generation changes the prefix. Training changes the parameter values.

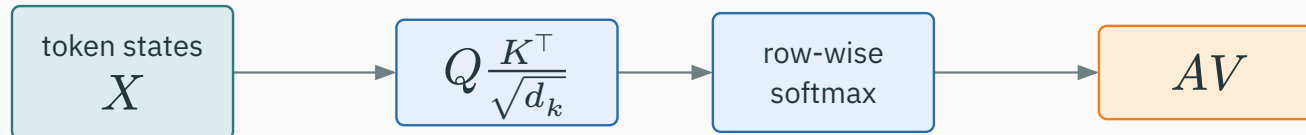
Next: combine several attention heads, add normalization and a feed-forward layer, and stack blocks. We will reuse this retained-input idea.

Lecture 3: From attention to the Transformer

Recall one attention head

Each token gets a weighted combination of the visible values

$$Q = XW_Q \quad K = XW_K \quad V = XW_V$$



One head makes one set of Q , K , and V vectors. Each row uses its own weights to combine the values it is allowed to see.

Why use more than one head?

Priya opened notebook ; she smiled

Several parts of this sentence could help us predict what comes after `smiled`.

NEARBY

What just happened?

WHO?

Which earlier person
does she refer to?

STRUCTURE

Where did one clause end
and another begin?

Can we learn a separate set of attention weights for each?

Give each token several attention heads

HEAD 1

what is nearby?

might use nearby tokens

HEAD 2

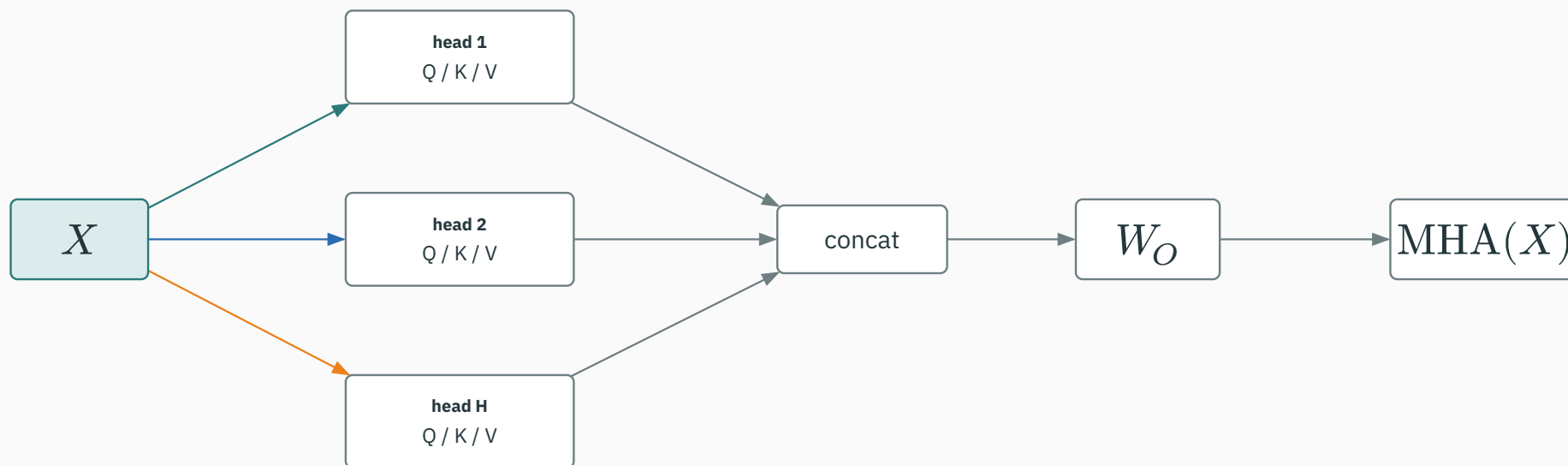
which person?

might connect distant tokens

HEAD 3

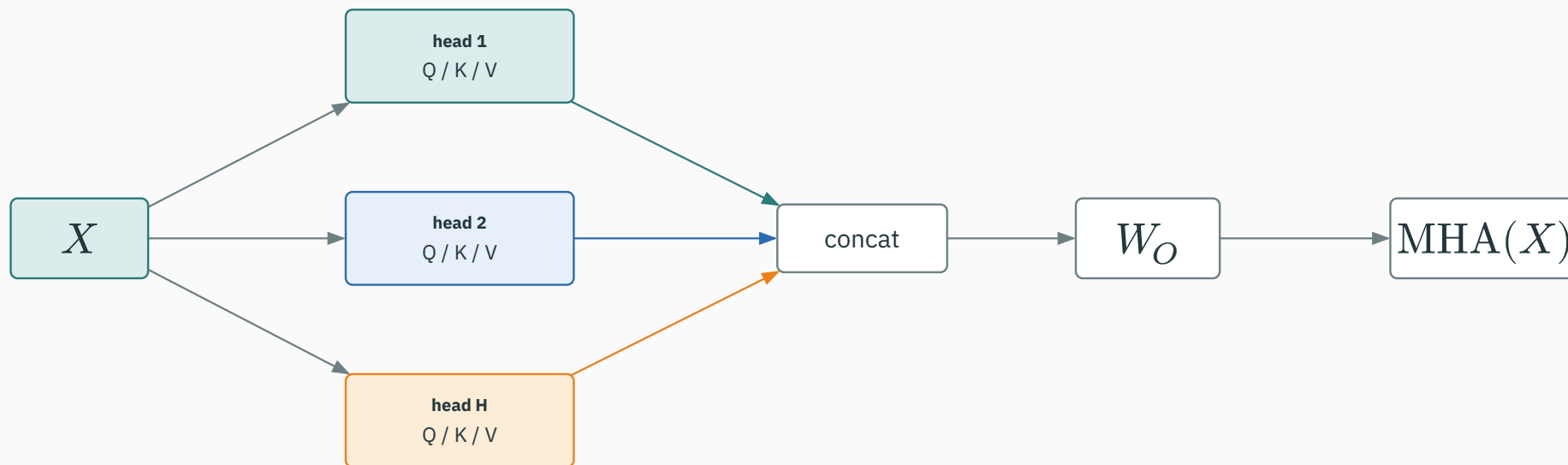
where is the boundary?

might use punctuation



Each head learns its own projections

$$Q_h = XW_Q^{(h)}, \quad K_h = XW_K^{(h)}, \quad V_h = XW_V^{(h)}$$



Every head receives the same X and learns its own weights

One head may look nearby

In this example, the row for `smiled` puts most of its weight on nearby `she`.

keys →	Priya	opened	notebook	;	she	smiled
smiled →	.05	.08	.07	.04	.66	.10

| Rows are queries; columns are source tokens and their keys.

| Softmax runs across the six key columns in this row. The weights sum to 1.

Another head may look back to a person

A second head might connect `she` with `Priya`. Here is one possible row of weights.

keys →	Priya	opened	notebook	;	she	smiled
she →	.61	.12	.10	.07	.10	0

| Rows are queries; columns are source tokens and their keys.

| `smiled` is in the future for this row. The causal mask makes its attention weight exactly zero.

A third head may notice a boundary

For a third head, the row for `smiled` might put most of its weight on the semicolon.

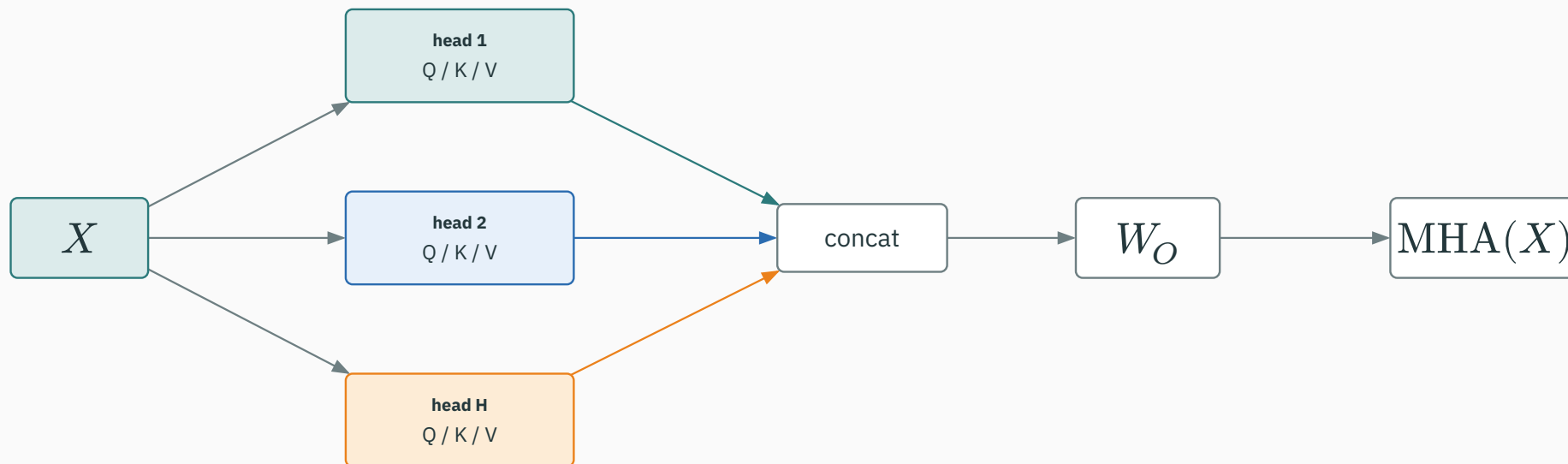
keys →	Priya	opened	notebook	;	she	smiled
smiled →	.07	.08	.09	.58	.13	.05

| Rows are queries; columns are source tokens and their keys.

| We chose these weights to illustrate possible patterns. Training learns the heads across many examples; a head need not have one clear role.

The heads do the same calculation separately

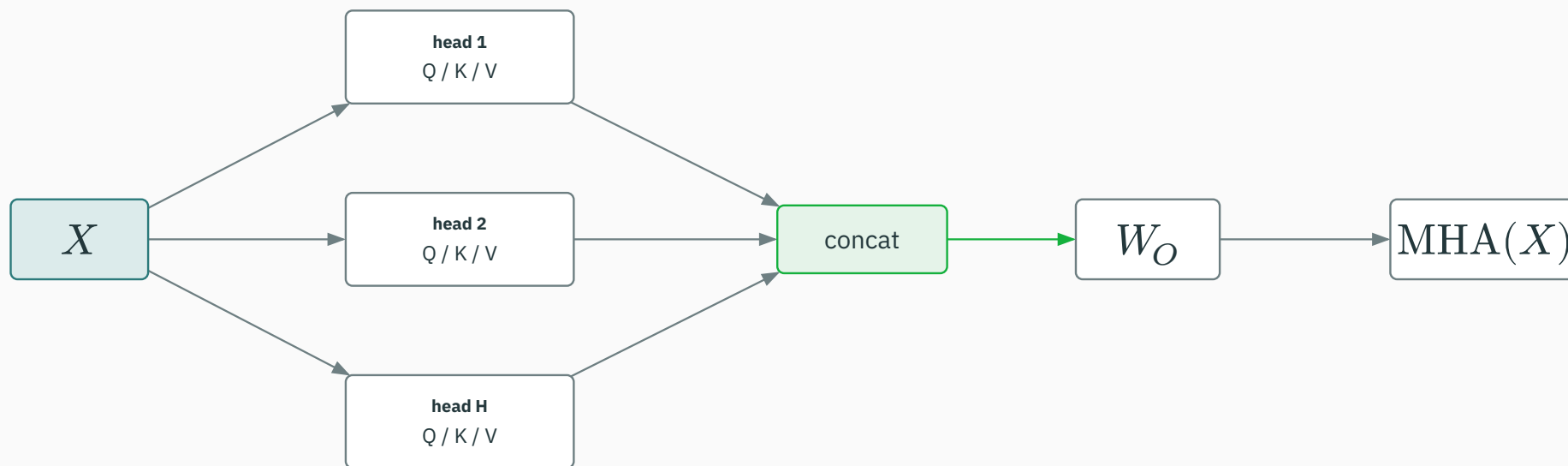
$$\text{head}_h = \text{Attention}(Q_h, K_h, V_h) = \text{softmax}\left(Q_h \frac{K_h^\top}{\sqrt{d_k}} + M\right) V_h$$



$$(Q_h : T \times d_k) \times (K_h^\top : d_k \times T) \rightarrow (A_h : T \times T) \times (V_h : T \times d_v)$$

$$\text{head}_h \in \mathbb{R}^{T \times d_v}$$

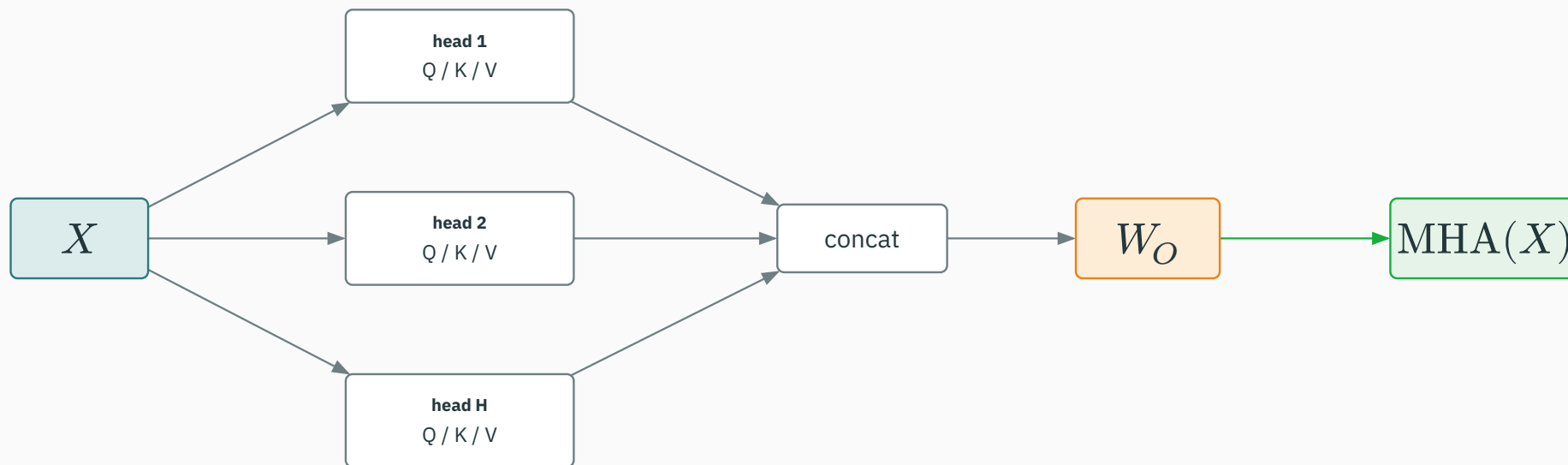
Put the head outputs side by side



$$\underline{C = \text{Concat}(\text{head}_1, \dots, \text{head}_H) \in \mathbb{R}^{T \times H d_v}}$$

For each token, write head 1's output, then head 2's output, and so on.
Concatenation keeps all the features from every head.

Mix the concatenated features with a linear layer



$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W_O$$

W_O is a learned linear layer. It combines features from the heads and produces width d_{model} , which we need for the residual addition.

Check the widths before moving on

COMMON WIDTH CHOICE

$$d_k = d_v = \frac{d_{\text{model}}}{H}$$

If $d_{\text{model}} = 512$ and $H = 8$,
then $d_k = d_v = 64$.

CONCATENATION CHECK

$$H d_v = 8 \times 64 = 512$$

so C already has width d_{model} before W_O mixes it.

Each head produces 64 features. Concatenating eight heads gives 512.

What can an attention map tell us?

INSPECT

Look at weights, outputs, gradients, and how interventions change behavior.

BE CAREFUL

A head may change roles across inputs. A large weight alone cannot prove causal importance.

TEST

Remove a component or change the input, then compare the model's behavior.

Attention weights help us inspect a model. Explaining its behavior needs further evidence.

Attention mixes rows; the FFN mixes features

ATTENTION

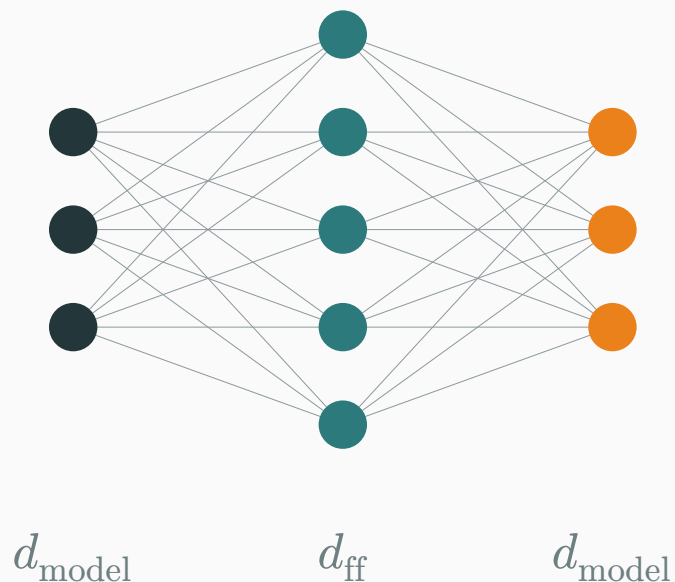
Mix information between visible token rows.

A block first mixes information across token rows, then transforms the features within each row.

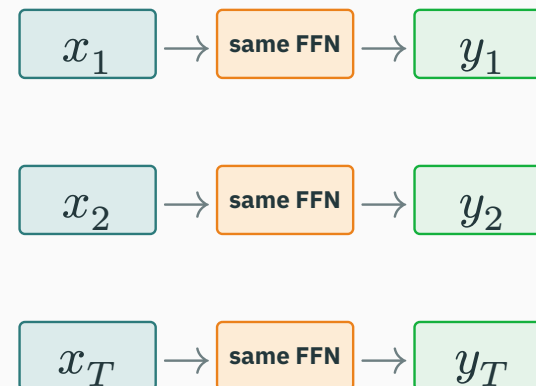
FEED-FORWARD NETWORK

Recombine the features inside each row with a nonlinear map.

Give every token the same small neural network



$$\text{FFN}(x) = W_2 \varphi(W_1 x + b_1) + b_2$$



The same MLP weights are used at every position

Across tokens, then within a token

MHA · ACROSS ROWS

	key 1	key 2	key 3
query 1	.7	.2	.1
query 2	.1	.7	.2
query 3	.2	.3	.5

token positions exchange information

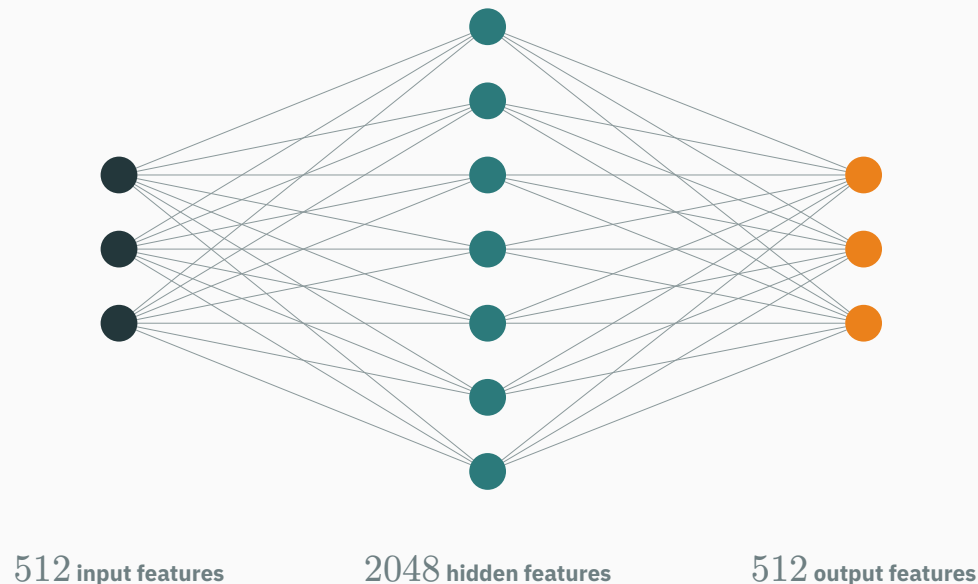
FFN · WITHIN ONE ROW



a nonlinear transformation of one row's features

Attention combines rows. The FFN transforms each row separately.

Why does the FFN widen in the middle?



$512 \rightarrow 2048 \rightarrow 512$: **compute more nonlinear features, then return to the original width.**

Can a sublayer pass its input through unchanged?

$$y = x + F(x)$$

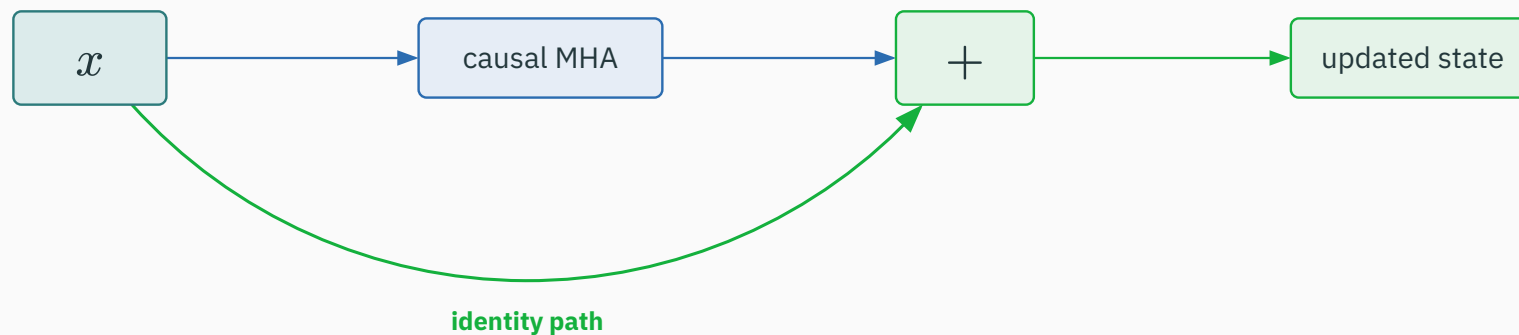
KEEP THE INPUT

If $F(x) \approx 0$, then $y \approx x$. The input passes straight through.

LEARN AN UPDATE

The sublayer learns the correction $F(x)$ to add to its input.

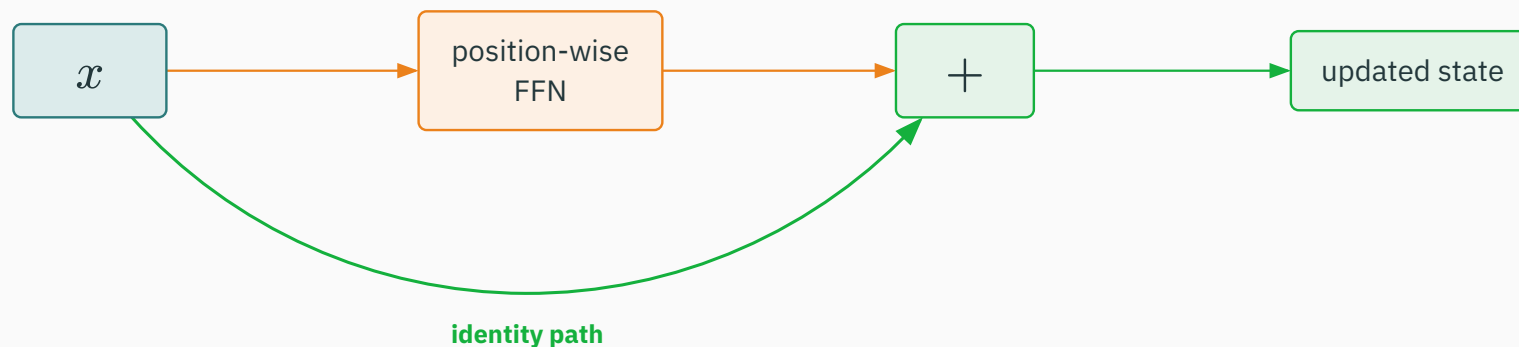
Add the attention output to its input



$$\underline{r = x + \text{MHA}(x)}$$

| Shapes must match: both x and $\text{MHA}(x)$ are $T \times d_{\text{model}}$.

Add the FFN output to its input

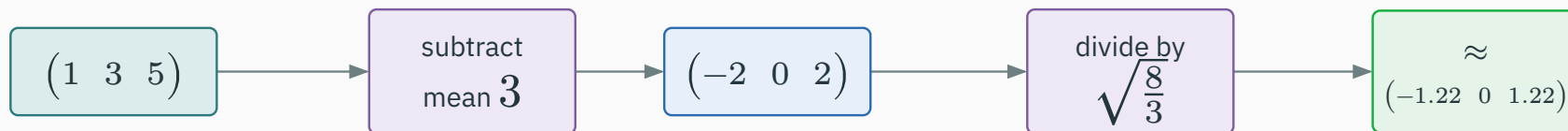


$$\underline{y = r + \text{FFN}(r)}$$

| The residual stream keeps width d_{model} through every sublayer and every block.

LayerNorm works across one token's features

Start with one token row. Subtract its mean, then divide by its standard deviation.



ONE ROW AT A TIME

GENERAL ROW x_i

$$\text{LN}(x_i) = \gamma \cdot \frac{x_i - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}} + \beta$$

Each token row has its own feature mean and variance. Learned γ, β are shared across positions.

Each token row is normalized separately

Where should LayerNorm go?

PRE-NORM

$$y = x + F(\text{LN}(x))$$

Normalize **before** the sublayer.

POST-NORM

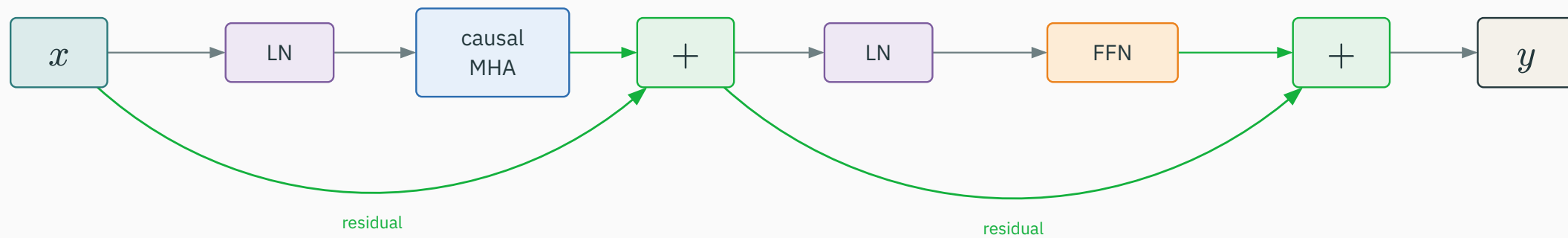
$$y = \text{LN}(x + F(x))$$

Normalize **after** residual addition.

We will use pre-norm in our block

Both versions are used. In the pre-norm drawing, we can follow the residual path around each sublayer.

Put the sublayers together



(same $T \times d_{\text{model}}$ stream)

(future positions masked)

(two residual additions)

attention → add input → FFN → add input

Write the block as two updates

ATTENTION SUBLAYER

$$u = x + \underbrace{\text{MHA}(\text{LN}(x))}_{\text{causal retrieval}}$$

FEED-FORWARD SUBLAYER

$$y = u + \underbrace{\text{FFN}(\text{LN}(u))}_{\text{per-token transformation}}$$

$x, u, y \in \mathbb{R}^{T \times d_{\text{model}}}$. The two green plus signs in the drawing are exactly the two additions above.

The block in five lines of PyTorch-style code

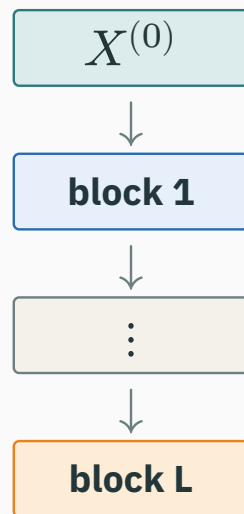
PyTorch

```
class Block(nn.Module):  
    def forward(self, x):  
        x = x + self.causal_attn(self.ln1(x))  
        x = x + self.ffn(self.ln2(x))  
        return x
```

For each sublayer: normalize its input, compute the output, and add the input back.

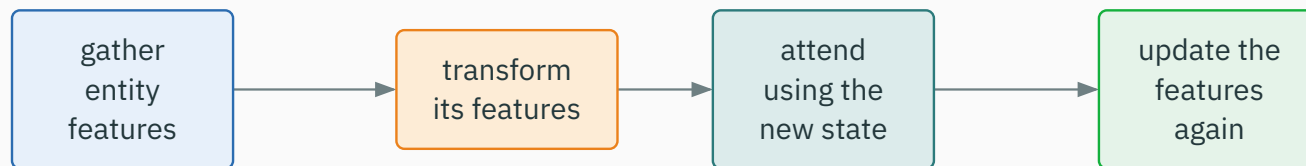
`self.causal_attn` wraps the attention calculation. We will add the full module, dropout, and initialization in the notebook.

Stack several blocks



Each block has its own attention, FFN, and LayerNorm parameters. The blocks share the same structure.

The next block uses the updated token states



FIRST BLOCK

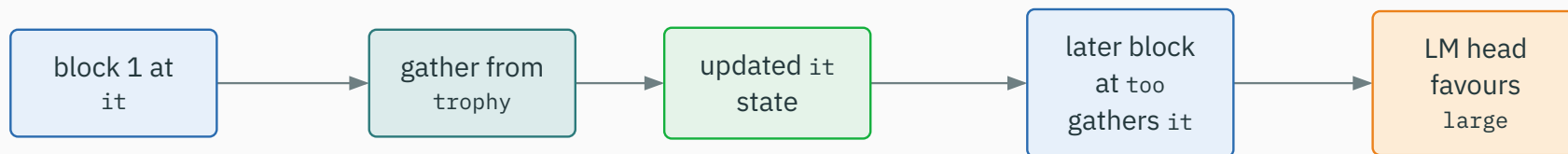
Might combine information from a name or a nearby verb in a token's state.

LATER BLOCK

Can use the updated state to compute a new set of attention weights.

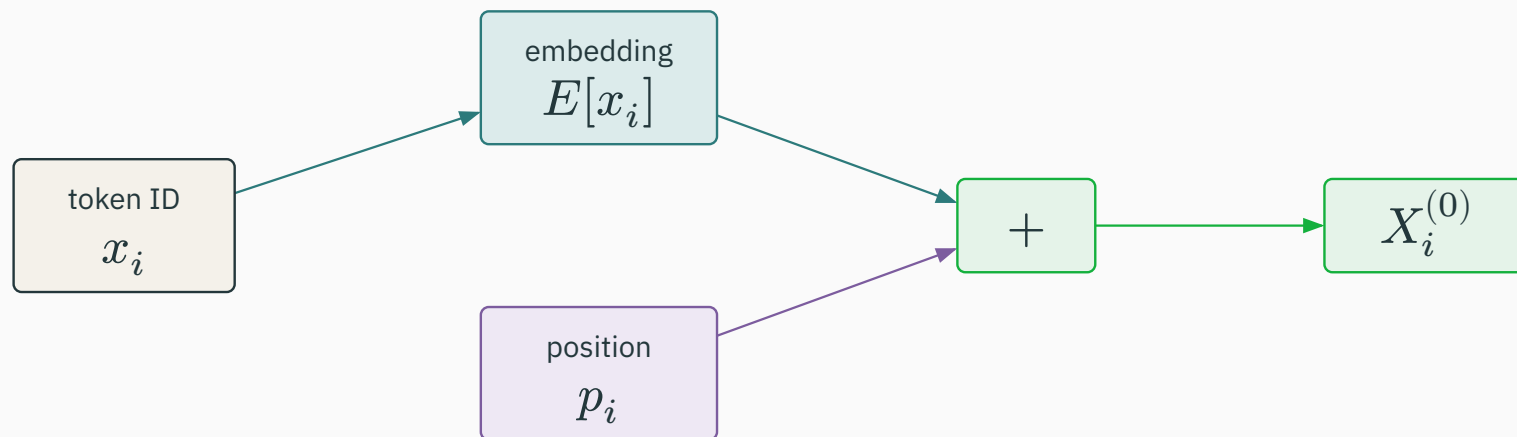
Several blocks allow several steps of computation. We cannot assume that each layer corresponds to one human reasoning step.

Information can pass through an intermediate token



The later block uses the updated state at it. This picture illustrates one possible computation.

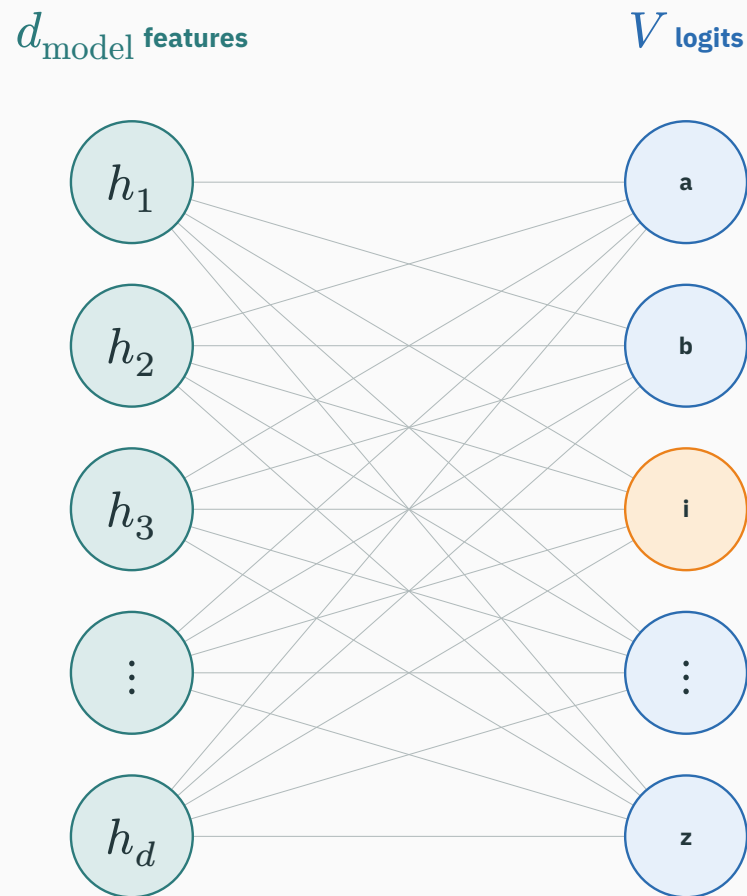
Begin with token and position information



$$X_i^{(0)} = E[x_i] + p_i$$

Without p_i , identical tokens begin with identical rows. Position information tells the model where each copy occurs. Some modern models add position information inside attention instead.

Turn the final token state into token scores



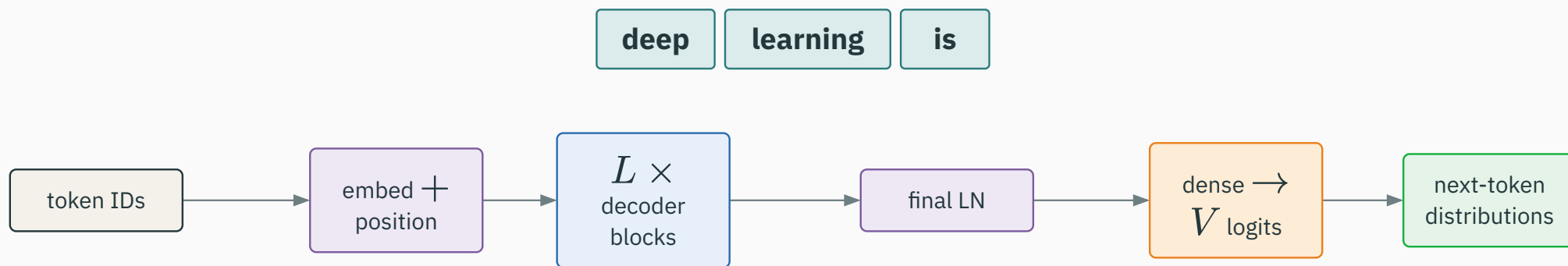
$$z_t = h_t W_{\text{vocab}} + b$$

$$p_t = \text{softmax}(z_t)$$

$$\begin{matrix} (h_t : d_{\text{model}}) \\ (z_t : V) \end{matrix}$$

softmax runs over the V vocabulary scores

The decoder-only model from end to end



The output is still a probability distribution over the next token.

Slide the targets one place to the left

	POSITION 1	POSITION 2	POSITION 3	POSITION 4
input	<BOS>	deep	learning	is
target	deep	learning	is	fun

Each row uses its prefix to predict the next token

During training, we know the whole sentence

DURING ONE TRAINING PASS

Feed in the true sentence. Row t can use the true prefix $x_{\leq t}$.

The causal mask hides future tokens from each row, so a row cannot see the answer it must predict.

ALL ROWS TOGETHER

Score the next token at every row in the same pass. Sampling and feeding a token back happens during generation.

Score every next-token guess

$$\mathcal{L} = - \sum_{t=1}^{T-1} m_t \log p_{\theta}(x_{t+1} \mid x_{\leq t})$$

LOOK ONLY LEFT

The causal mask keeps future words hidden.

SCORE REAL ROWS

m_t skips padding or ignored positions.

SCORE THE TARGET

Cross-entropy uses the probability of the token one step to the right.

Code often averages over valid tokens. The sum above shows one loss term for each next-token prediction.

The next-token objective stays the same

FIXED-WINDOW CHARACTER MODEL

context: a a b

target: i

loss: $-\log p_{\theta}(i \mid a, a, b)$

DECODER TRANSFORMER

context: $x_{\leq t}$

target: x_{t+1}

loss: $-\log p_{\theta}(x_{t+1} \mid x_{\leq t})$

Both models minimize the loss on the next token

Shift the targets and compute cross-entropy

PyTorch

```
inputs  = tokens[:, :-1]
targets = tokens[:, 1:]
logits  = model(inputs)
loss = F.cross_entropy(
    logits.flatten(0, 1), targets.flatten())
```

The two slices pair each input position with its next-token target.

For padded batches, pass the padding token ID as `ignore_index`. We will add the data loader and masks in the notebook.

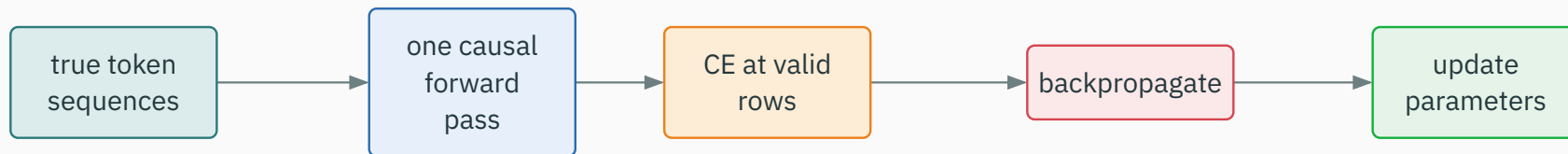
Training: one pass, many next-token guesses

TRAIN tokens $\rightarrow$ logits $\rightarrow$ loss $\rightarrow$ update

GENERATE prefix $\rightarrow$ distribution $\rightarrow$ choose

EVALUATE held-out tokens $\rightarrow$ mean NLL / PPL

$\rightarrow$ append



Positions are processed in parallel during the training forward pass. Layers are still evaluated one after another.

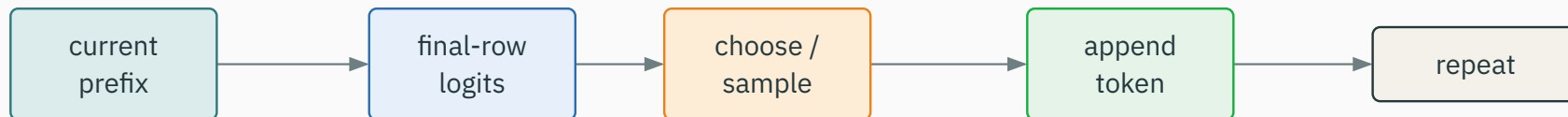
Generation: one new token, then go again

TRAIN tokens $\rightarrow$ logits $\rightarrow$ loss $\rightarrow$ update

GENERATE prefix $\rightarrow$ distribution $\rightarrow$ choose

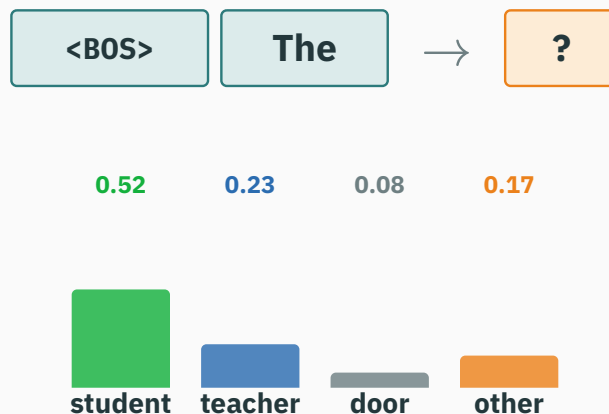
EVALUATE held-out tokens $\rightarrow$ mean NLL / PPL

$\rightarrow$ append



Autoregressive generation is sequential because the next model input depends on the token just chosen.

Generation, first step



choose student → append it to the prefix

| Use the distribution at the final input position to choose the next token.

Generation, one step later



0.47 0.25 0.16 0.12



choose opened → continue with a longer prefix

The prefix now has one more token. We can cache calculations from the earlier positions and reuse them.

The whole generation loop

PyTorch

```
for _ in range(max_new_tokens):  
    logits = model(ids)[: , -1]  
    probs = logits.softmax(dim=-1)  
    next_id = torch.multinomial(probs, 1)  
    ids = torch.cat([ids, next_id], dim=1)
```

Score the prefix, sample one token, append it, and repeat.

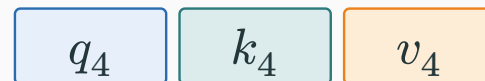
Greedy decoding would use `argmax` instead of sampling. A real implementation also handles stopping tokens and the KV cache.

Cache the keys and values from earlier positions

ALREADY CACHED AT EACH LAYER



NEW TOKEN



Score q_4 against cached $K_{1:3}$ and new k_4 ; append k_4, v_4 to the cache.

In causal incremental decoding, adding a future token leaves earlier hidden rows unchanged. Their projected keys and values stay the same too.

What does a longer context cost?

TRAINING SCORE GRID

	k_1	k_2	k_3	k_4
q_1	•	×	×	×
q_2	•	•	×	×
q_3	•	•	•	×
q_4	•	•	•	•

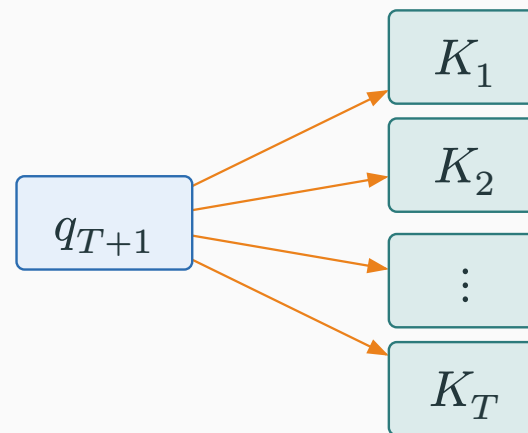
$\Theta(T^2)$ pair scores

DECODING CACHE

K_1	V_1
K_2	V_2
$\vdots$	$\vdots$
K_T	V_T

stored rows grow with T

ONE NEW QUERY

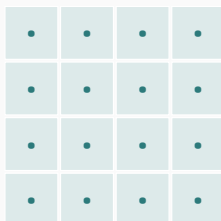


compare with all cached keys

Optimized kernels can avoid materializing the full $T \times T$ matrix, but they do not remove the vanilla all-pairs arithmetic.

Three common ways to arrange Transformer blocks

encoder-only



VISIBILITY

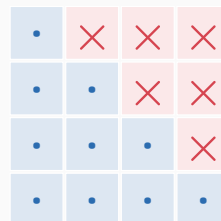
all tokens can see both directions

COMMON OBJECTIVE

masked-token / representation learning

Example: BERT

decoder-only



VISIBILITY

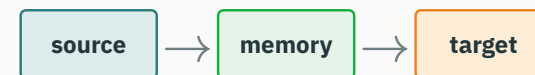
each token sees only its prefix

COMMON OBJECTIVE

next-token prediction

Example: GPT-style LM

encoder-decoder



VISIBILITY

source bidirectional; target causal

COMMON OBJECTIVE

conditioned sequence generation

Example: translation

Encoder-only: read the whole input

keys →	the	river	bank	was	wide
bank →	.12	.39	.18	.10	.21

VISIBILITY

Every position may attend left and right; there is no causal triangle.

TYPICAL USE

Build contextual features for classification, tagging, retrieval, or masked-token training.

| BERT uses an encoder. The next-token model we built uses a causal decoder.

Decoder-only: read only the prefix

keys →	<BOS>	The	cat	sat	on
sat →	.08	.14	.43	.35	0

VISIBILITY

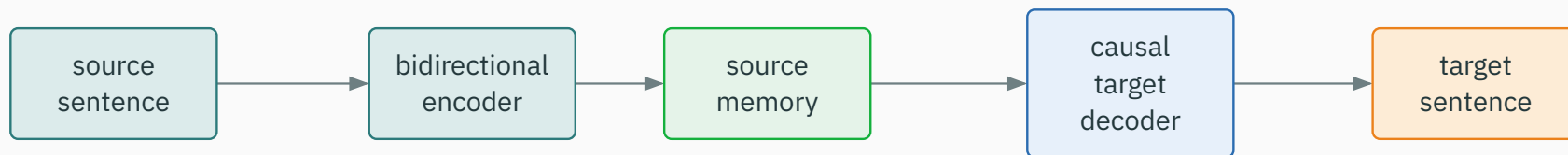
Position t attends only to positions $\leq t$.

OBJECTIVE

Predict x_{t+1} from $x_{\leq t}$ at every valid row.

This is the decoder we have been building.

Encoder-decoder: read one sequence, write another



The decoder uses causal self-attention over the generated target **and** cross-attention to the encoded source. Translation is the classic example.

Self-attention and cross-attention use the same calculation

SELF-ATTENTION

$$Q = XW_Q$$

$$K = XW_K \quad V = XW_V$$

Queries, keys, and values come from the same sequence.

CROSS-ATTENTION

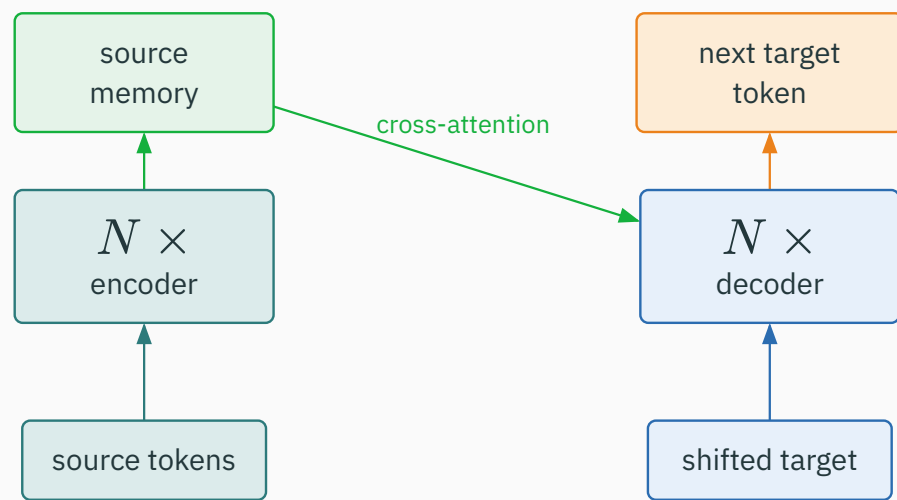
$$Q = Y_{\text{decoder}}W_Q$$

$$K = X_{\text{source}}W_K \quad V = X_{\text{source}}W_V$$

Queries come from the decoder. Keys and values come from the encoded source.

The attention calculation stays the same. What changes is the source of the queries, keys, and values.

The 2017 Transformer had an encoder and a decoder



2017

Vaswani et al., **Attention Is All You Need**, introduced the Transformer as an encoder-decoder model for sequence transduction.

We focused on a decoder-only model because it directly extends next-token prediction.

Vaswani et al. (2017) · arXiv:1706.03762

Put the sequence models side by side

model	available context	path to far token	training across time	main trade-off
fixed-window MLP	exactly k previous tokens	1 within window; unavailable outside	parallel examples	input width and parameters tied to k
RNN	prefix compressed into recurrent state	$O(T)$ recurrent steps	sequential recurrence	compact state; long dependency path
temporal convolution	finite receptive field grows with depth/dilation	depends on kernel and depth	parallel	local bias; depth needed for reach
self-attention	all visible tokens in the mask	1 attention edge per layer	parallel masked rows	vanilla all-pairs score cost

The path length counts the steps information must take through the model. A short path does not guarantee that the model learns the dependency.

Longer memory has a price

FIXED MLP

Uses a fixed window k . A larger context needs a different first layer.

RNN

Roughly linear steps in T , but those recurrent steps are sequential.

SELF-ATTENTION

Vanilla pair scores are $\Theta(T^2)$; a causal triangle is still $\Theta(T^2)$.

Compare both what context a model can use and what that access costs.

These complexity terms omit width, layers, constants, kernels, and hardware.
Each architecture has trade-offs.

What does attention give us?

GIVES

CONTENT-DEPENDENT ROUTING

A query can weight visible sources differently for each token and layer.

SHORT COMMUNICATION PATHS

Any visible token can directly contribute within an attention sublayer.

DOES NOT GUARANTEE

TRUTH OR REASONING

The model may still generate false or poorly justified text.

EXPLANATION OR UNLIMITED MEMORY

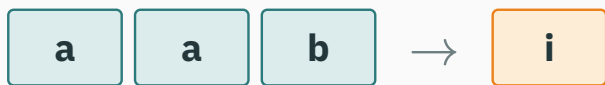
Attention weights are not explanations. Context length and the KV cache remain finite.

Four quick checks

CLAIM	CORRECTION
“Attention weights are explanations.”	We can inspect the weights, but an explanation needs stronger causal evidence.
“One head learns one linguistic rule.”	A pattern may be spread across heads and may change across inputs.
“Masking is only used at generation.”	We also need the causal mask during parallel next-token training.
“A Transformer removes the MLP.”	Every standard block contains a substantial position-wise FFN.

Compare our two language models

FIXED-WINDOW MLP



DECODER TRANSFORMER



Both use IDs → embeddings → logits → cross-entropy → sampling

The MLP concatenates three embedding rows. The Transformer keeps one row per token and combines information between rows through causal attention.

Can you finish this sentence?

“A Transformer language model differs from our character MLP mainly because ...”

...each position can weight earlier tokens differently, combine their values, pass the result through an MLP, and repeat this in the next block.

We still train it to predict the next token from the previous tokens.

Optional calculations

Appendix: differentiate the softmax mixture

$$\alpha_j = \frac{\exp(s_j)}{\sum_l \exp(s_l)}, \quad h = \sum_j \alpha_j v_j$$

$$\Delta e = h W_O, \quad e' = e + \Delta e, \quad g_h = g_{e'} W_O^\top$$

$$\frac{\partial \alpha_j}{\partial s_l} = \alpha_j (\delta_{jl} - \alpha_l)$$

$$g_{s_j} = \alpha_j (g_{\alpha_j} - \sum_l \alpha_l g_{\alpha_l})$$

Substitute $g_{\alpha_j} = g_h \cdot v_j$. Since $\sum_l \alpha_l v_l = h$, the whole Jacobian reduces to:

$$g_{s_j} = \alpha_j g_h \cdot (v_j - h)$$

$$g_q = \sum_j g_{s_j} \frac{k_j}{\sqrt{d_k}}, \quad g_{k_j} = g_{s_j} \frac{q}{\sqrt{d_k}}$$

Row vectors; e, q, h omit the fixed receiver's index. g is a loss gradient; lowercase δ is an index indicator, not the update Δe .

Appendix: the matrix backward pass

X stacks input embeddings; X' stacks updated embeddings. $P, Y: T \times 8$ probabilities / one-hot targets; $T = 6$.

$$H = AV, \quad \Delta X = HW_O, \quad X' = X + \Delta X, \quad Z = X'U + b$$

PREDICTION AND RESIDUAL ADDITION

$$\begin{aligned} g_Z &= (P - Y) \div T \\ g_U &= (X')^\top g_Z, \quad g_b = \sum_i (g_Z)_{i,:} \\ g_{X'} &= g_Z U^\top \\ g_{\Delta X} &= g_{X'} \\ g_{W_O} &= H^\top g_{\Delta X} \\ g_H &= g_{\Delta X} W_O^\top \end{aligned}$$

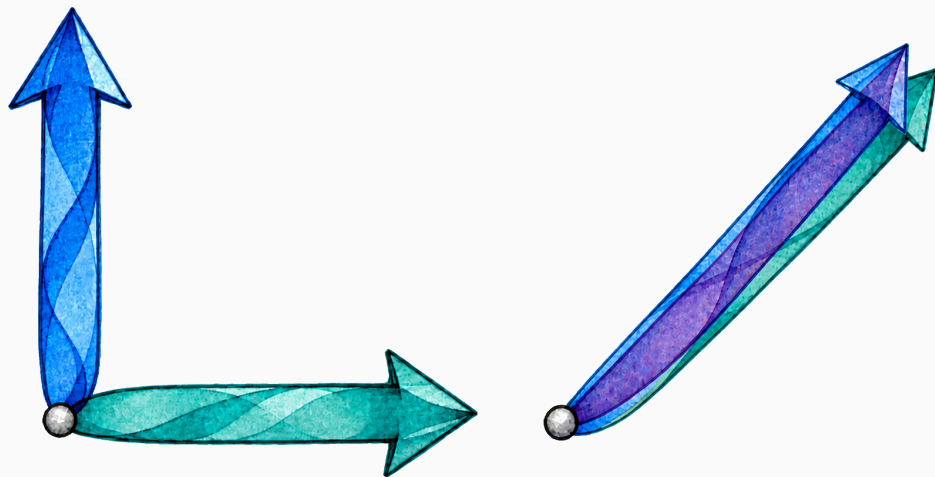
MIXTURE, SCORES, AND PROJECTIONS

$$\begin{aligned} g_V &= A^\top g_H, \quad g_A = g_H V^\top \\ &\text{Row-wise softmax backward gives } g_S. \\ &\text{Masked score gradients are zero.} \\ g_Q &= (g_S K) \div \sqrt{d_k} \\ g_K &= (g_S^\top Q) \div \sqrt{d_k} \\ g_{W_Q} &= X^\top g_Q; \text{ similarly for } K, V. \end{aligned}$$

$$g_X = g_{X'} + g_Q W_Q^\top + g_K W_K^\top + g_V W_V^\top$$

Scatter-add g_X into the matching token rows of E and position rows of R . Shared parameters accumulate gradients.

Appendix: what does the dot product notice?



$$q \cdot k = \|q\| \|k\| \cos(\theta)$$

Direction and magnitude both matter.

$$q = (1 \ 0), \quad k = (0 \ 1)$$

$$q \cdot k = 0$$

$$q = (1 \ 0), \quad k = (2 \ 0)$$

$$q \cdot k = 2$$

A token's query and key are different projections. Its own key need not receive the largest score, even when vectors are normalized.

References and lecture plan

References

NEXT-TOKEN PREDICTION

Nipun Batra · Next Token Prediction

aabid, embeddings, concatenation, classification, sampling

LANGUAGE MODELS

Stanford CME 295 · Lecture 1: Transformer

language tasks, tokenization, and the full Transformer

SEQUENCE MODELS

MIT 6.S191 · Deep Sequence Modeling

queries, keys, values, and self-attention

THE ORIGINAL TRANSFORMER

Vaswani et al. · Attention Is All You Need

scaled dot-product attention and the encoder-decoder Transformer

3Blue1Brown · Attention: contextual updates, query/key matching, and value messages.

Technical diagrams were drawn in Typst/Fletcher. Illustrations were generated with OpenAI image generation; labels were added in Typst. Schematic token splits and attention maps are labeled as teaching examples.

The three lectures

- Lecture 1** Text $\rightarrow$ tokens $\rightarrow$ next-token prediction
- Lecture 2** Context $\rightarrow$ weighted averages $\rightarrow$ attention
- Lecture 3** Multi-head attention + MLP + residual paths $\rightarrow$ Transformer

SLIDES

Use the presentation PDF to reveal each step. The handout shows completed slides.

LECTURE PLAN

The Markdown outline records examples, questions, and suggested stopping points.